

LABMENTIX

Cybersecurity Training Program

CYBERSECURITY PROJECT REPORT

15-Day Hands-On Project Submission

Field	Details
Project Title	Cryptographic Cipher Analyzer – a Caesar and Vigenere cryptanalysis tool
Student Name	Malek Minhaz
Batch / Cohort	Batch 01 – June 2026
Domain	Cybersecurity – Cryptography and Cryptanalysis
Trainer Name	Ganesh Kuche
Submission Date	21/09/2026
Difficulty Level	Intermediate

A command-line tool that implements the Caesar and Vigenere ciphers and demonstrates, step by step, how both of them are broken.

CONTENTS

Contents

1. Project Abstract
2. Introduction – background, problem statement and objectives
3. Tools and Environment Setup – tools used and the lab description
4. Methodology – fourteen numbered steps with screenshots (Figures 1-52)
5. Results and Observations – findings table and analysis
6. Challenges Faced
7. Learning Outcomes
8. Conclusion and Future Scope
9. References

Student Declaration

SECTION 1 – PROJECT ABSTRACT

1. Project Abstract

This project delivers the Cryptographic Cipher Analyzer, a Python command-line application that implements the two most widely taught classical ciphers – the Caesar cipher and the Vigenere cipher – and then attacks them with the same techniques a real cryptanalyst would use. Textbooks state that these ciphers are weak, but the weakness only becomes convincing when the learner watches a ciphertext that looked unreadable collapse into plain English in front of them. The tool was therefore built around that moment: every operation prints not only the result but also the evidence that justifies it, and never presents a statistical guess as a proven fact.

The application provides direct encryption and decryption for both ciphers, single-letter frequency analysis, an automated Caesar breaker that scores all twenty-six shifts with the chi-square statistic, an Index of Coincidence analyser, a Kasiski examination of repeated sequences, a combined Vigenere cryptanalysis workflow that recovers the keyword one column at a time, file based encryption and decryption, a security assessment of each algorithm, and timestamped text reports. It offers both an interactive menu and a scriptable command line, and it is covered by 163 automated tests that all pass.

Running the tool on the supplied sample ciphertexts produced concrete results: the Caesar sample was broken automatically and returned the correct shift of 3 with the plaintext "HELLO WORLD / CRYPTOGRAPHY IS THE PRACTICE AND STUDY OF TECHNIQUES FOR SECURE COMMUNICATION...", and the Vigenere sample was analysed to an estimated key length of 5 and cracked to the keyword LEMON, again as the first ranked candidate. The exercise confirms the central lesson of the course: a cipher is only as strong as the secrecy of its key, and a large keyspace is worthless if the ciphertext still leaks statistical structure.

SECTION 2 – INTRODUCTION

2. Introduction

2.1 Background

Cryptography is the practice and study of techniques for secure communication in the presence of adversaries. Its classical branch covers the ciphers that were used before computers existed, and although they are far too weak to protect modern data, they remain the clearest way to learn how cryptographic thinking works. The Caesar cipher, used by Julius Caesar around 58 BC, replaces every letter with another letter a fixed number of positions further along the alphabet, which gives an attacker only twenty-six keys to try. The Vigenere cipher, described in the sixteenth century, removes that weakness by applying a different Caesar shift for every letter using a repeating keyword, and was considered unbreakable for around three hundred years. Studying why the first cipher falls to trivial brute force while the second falls to statistics rather than to brute force teaches the difference between key size and real security – the same distinction that separates a modern algorithm such as AES from a large but badly designed key.

2.2 Problem Statement

Classical cryptography is usually taught as theory. Students are told that the Caesar cipher can be brute forced and that the Vigenere cipher leaks its key length through repeated sequences, but they rarely run the attack on real text, choose a key length themselves, watch a wrong guess produce nonsense, and correct it. There is also no single tool that shows the whole attack chain in one place: ordinary calculators or spreadsheet scripts compute letter frequencies, but they do not connect the frequency profile to the Index of Coincidence, to the Kasiski factors, to the per-column key recovery and finally to the recovered keyword. Without that connection the learner cannot see which evidence is decisive and which is merely suggestive, and it is easy to leave the exercise believing that a statistical estimate is a proof. This project addresses that gap with one self-contained tool that performs the complete cryptanalysis of both ciphers and labels clearly which values are confirmed and which are estimates.

2.3 Objectives

The project set out to achieve the following specific goals:

- To implement the Caesar and Vigenere ciphers in Python with correct handling of punctuation, spaces, digits and case.
- To build a single-letter frequency analysis module and compare its output against standard English letter frequencies.
- To automate the breaking of the Caesar cipher by testing all twenty-six shifts and ranking the results with the chi-square statistic.
- To implement the Index of Coincidence and Kasiski examination in order to estimate the length of a repeating keyword.
- To recover a Vigenere keyword column by column, verify the candidate by reading the decrypted text, and report the result as an unconfirmed estimate.
- To add file based encryption and decryption together with a security (strength) assessment of both algorithms, and to cover the whole tool with automated tests and generated text reports.

SECTION 3 – TOOLS & ENVIRONMENT

3. Tools and Environment Setup

The whole project was built and demonstrated on a single Windows machine using only the Python standard library, so it can be reproduced anywhere Python 3.10 or later is installed. The table below lists every tool that was used and the part it played.

Tool / Software	Version / Platform	Purpose
Python	3.13.14 (Windows 64-bit)	Core language runtime; the tool itself imports only the standard library
Windows 11	64-bit	Development and demonstration operating system
Windows Terminal (PowerShell)	Windows 11 bundled	Running the tool and capturing the screenshots in Section 4
GNU Bash (Git Bash)	5.x	Helper file commands used in the walkthrough (ls, head, cat)
unittest	Python standard library	Automated test suite – 163 tests covering ciphers, analysis and files
python-docx	1.2	Building this report and inserting the figures programmatically
Pillow (PIL)	12.3	Rendering the console screenshots that appear in Section 4

3.1 Lab / Environment Description

No virtual machines or cloud instances were required, because the project is a self-contained Python application rather than a network or host based attack. The lab consisted of one Windows 11 workstation with Python 3.13.14 installed as the system interpreter. The project lives in a single folder, `crypto_cipher_analyzer`, which contains the entry point `main.py`, the packages `analysis` (cryptanalysis algorithms), `ciphers` (Caesar and Vigenere implementations), `file_operations` (safe file input and output) and `utils` (terminal presentation, validation, text helpers and report generation), plus the tests and `sample_data` folders and the `reports` folder that receives the generated text reports.

The tool can be driven in two ways. Interactive mode (`python main.py`) presents a numbered menu and never crashes on unexpected input; command line mode runs one operation per invocation, for example `python main.py caesar-encrypt --text "HELLO" --shift 3`, which makes the tool easy to script and easy to demonstrate during a viva. Every analysis command accepts `--report` to write a timestamped text file under `reports/`, and every file operation refuses to overwrite an existing file unless `--overwrite` is given, so the original data is never lost. All output shown in the next section was captured from this environment exactly as the user sees it; the only post-processing was re-wrapping long lines at the width of the window shown in each figure.

SECTION 4 – METHODOLOGY / STEP-BY-STEP PROCESS

4. Methodology

This section documents exactly how the project was built and run. Each step states what was done and why, shows the command that was executed, and is followed by the resulting console screenshot with a caption describing what the screenshot shows. 52 screenshots are included as Figures 1 to 52 and they were all captured from real runs of the finished tool on the sample data supplied with the project. A screenshot that is taller than half a page is placed as two consecutive figures so that the pages of this section stay full and every line of the console stays legible. Nothing below is illustrative: the numbers, rankings and recovered keys are the values the tool actually printed.

Convention used in this section: values that the tool can prove (the Caesar shift of a decryption, the contents of a decrypted file) are reported as results, while values that depend on statistics (an estimated Vigenere key length, a ranked candidate key) are labelled as estimates and were verified separately by reading the decrypted text.

Step 1 – Setting up the environment and verifying the project structure

The workstation already had Python installed, so the first task was to record the exact interpreter version and to confirm the project layout before touching any code. The tool was deliberately written against the standard library only (argparse, dataclasses, pathlib, re, statistics), which means there is no dependency to install and no risk of a package upgrade changing the behaviour of the demonstrations.

```
python --version
python -c "import sys; print('Environment:', sys.executable)"
cd crypto_cipher_analyzer
ls -l
ls -l analysis ciphers file_operations utils
```

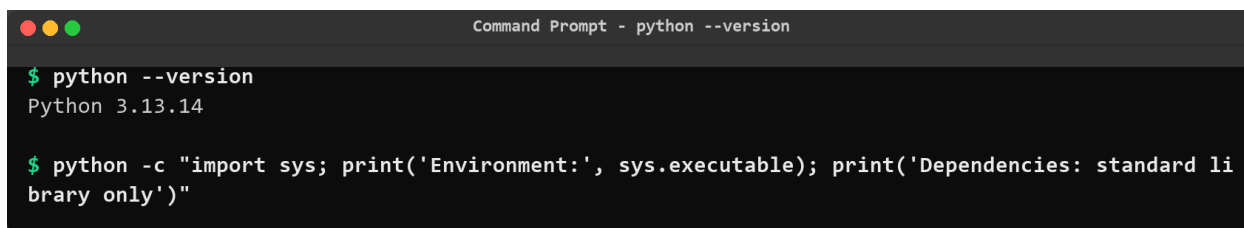
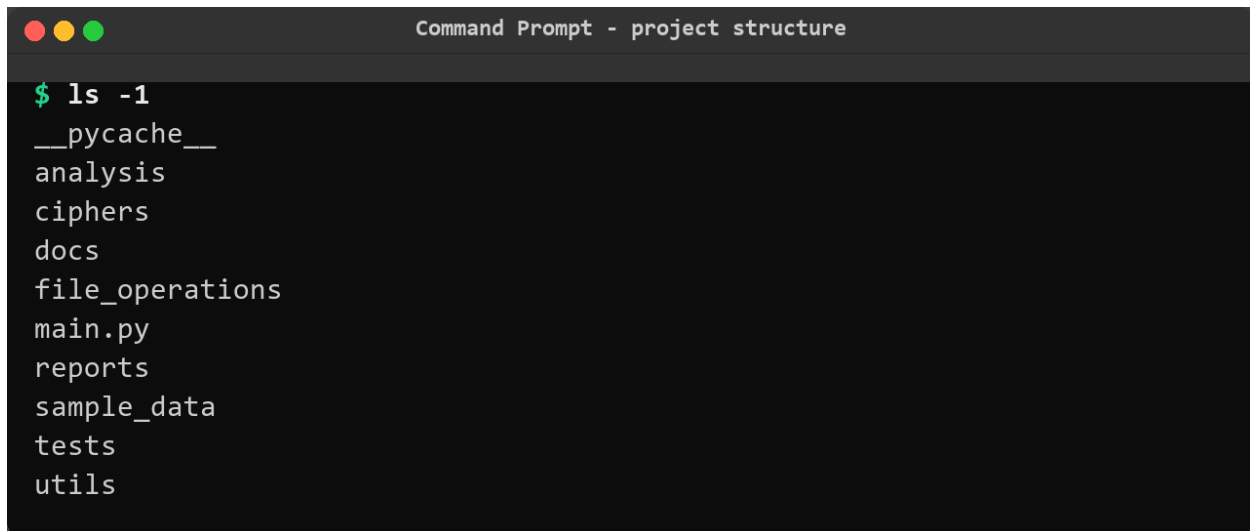


Figure 1: Environment check. Python 3.13.14 runs the tool and the project reports that it depends on the standard library only, so no third party package has to be installed to reproduce the results.

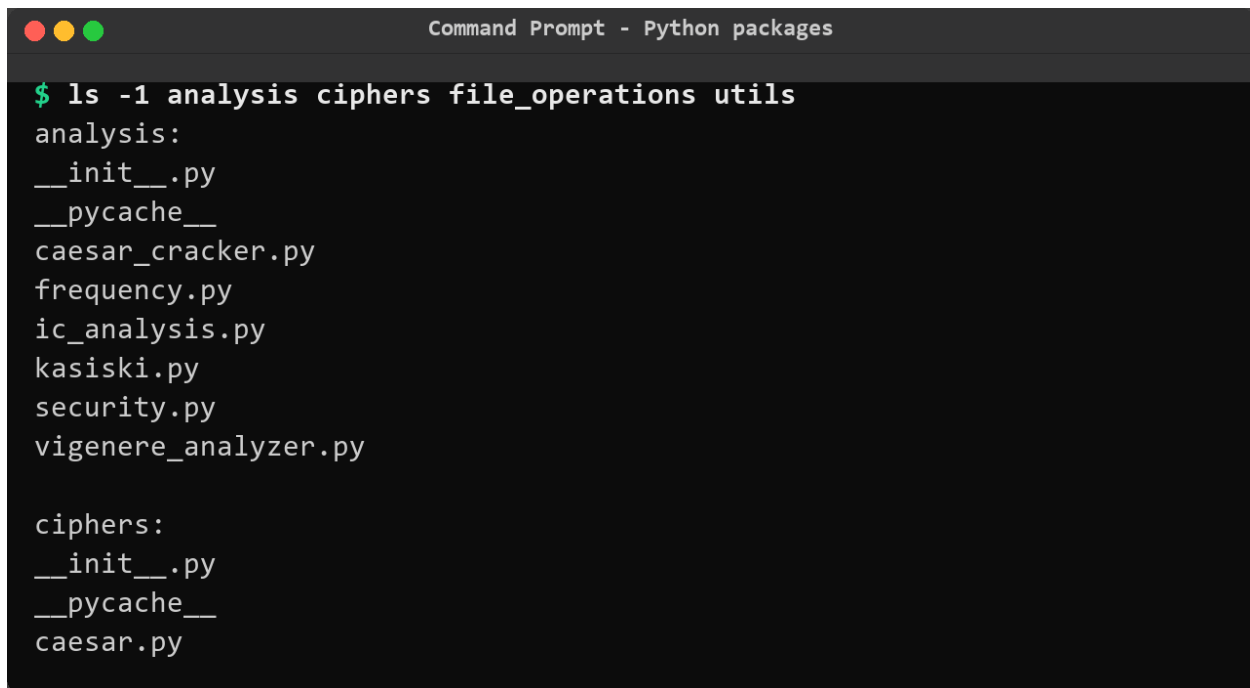
The listing below shows the top level of the project and then the contents of the four Python packages. Keeping the cryptanalysis algorithms (analysis), the ciphers themselves (ciphers), the disk access (file_operations) and the presentation helpers (utils) in separate packages is what allows the same algorithms to be driven from the interactive menu, from the command line and from the test suite without any duplication.



```
Command Prompt - project structure

$ ls -l
__pycache__
analysis
ciphers
docs
file_operations
main.py
reports
sample_data
tests
utils
```

Figure 2: Top level of `crypto_cipher_analyzer`. Besides `main.py`, the entry point, there are separate packages for the ciphers, the cryptanalysis algorithms, file handling, utilities, tests and sample data.

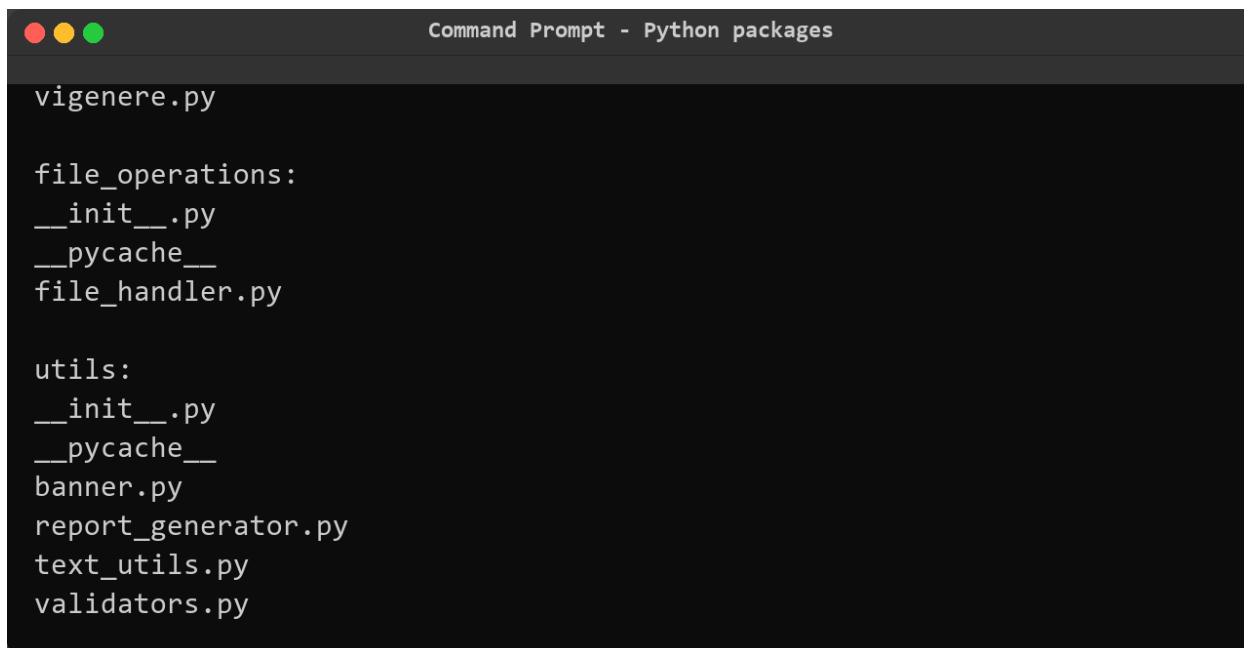


```
Command Prompt - Python packages

$ ls -l analysis ciphers file_operations utils
analysis:
__init__.py
__pycache__
caesar_cracker.py
frequency.py
ic_analysis.py
kasiski.py
security.py
vigenere_analyzer.py

ciphers:
__init__.py
__pycache__
caesar.py
```

Figure 3: Contents of the four Python packages: `analysis` holds the Caesar cracker, frequency, Index of Coincidence, Kasiski and Vigenere analysers; `ciphers` holds the two cipher implementations; `file_operations` holds the safe file reader and writer; `utils` holds the terminal banner, validators, text helpers and report generator.



```
Command Prompt - Python packages

vigenere.py

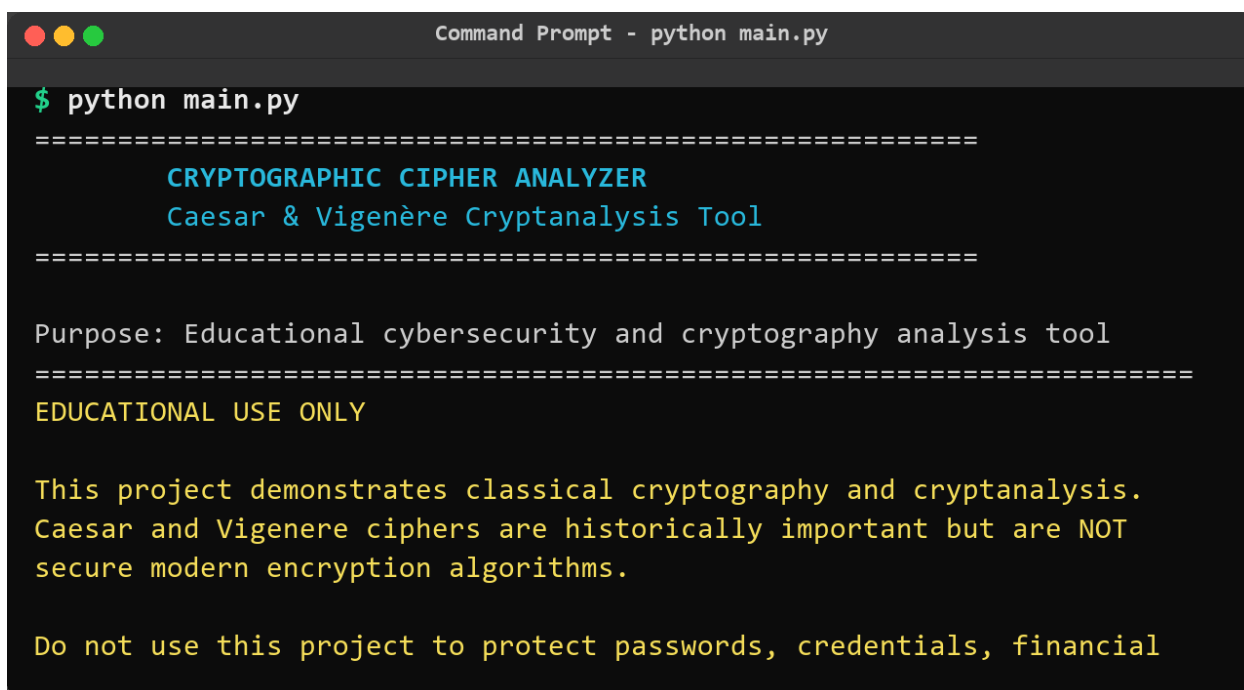
file_operations:
__init__.py
__pycache__
file_handler.py

utils:
__init__.py
__pycache__
banner.py
report_generator.py
text_utils.py
validators.py
```

Figure 4: Contents of the four Python packages: analysis holds the Caesar cracker, frequency, Ind... (continued).

Step 2 – Launching the tool and reading the main menu

Running main.py with no arguments starts interactive mode. The banner fixes the purpose of the tool at the top of the screen, the educational-use disclaimer is printed before any result is produced and pauses with a full stop between them, and the nine-entry menu exposes every capability of the program. Option 9 exits the tool cleanly, which is what was done here to capture the complete start-up screen in a single figure.



```
Command Prompt - python main.py

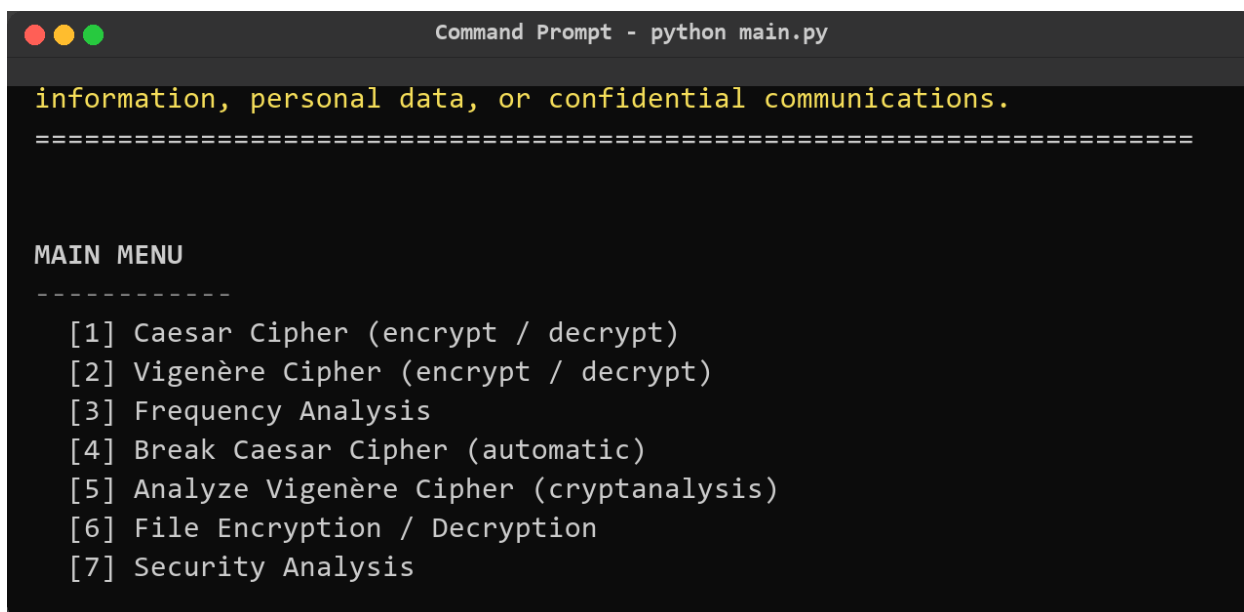
$ python main.py
=====
      CRYPTOGRAPHIC CIPHER ANALYZER
      Caesar & Vigenère Cryptanalysis Tool
=====

Purpose: Educational cybersecurity and cryptography analysis tool
=====
EDUCATIONAL USE ONLY

This project demonstrates classical cryptography and cryptanalysis.
Caesar and Vigenere ciphers are historically important but are NOT
secure modern encryption algorithms.

Do not use this project to protect passwords, credentials, financial
```

Figure 5: Start-up screen. The banner identifies the tool, the disclaimer states that the classical ciphers must not be used for real data, and the main menu lists the eight operations plus Exit; the session was ended with option 9.

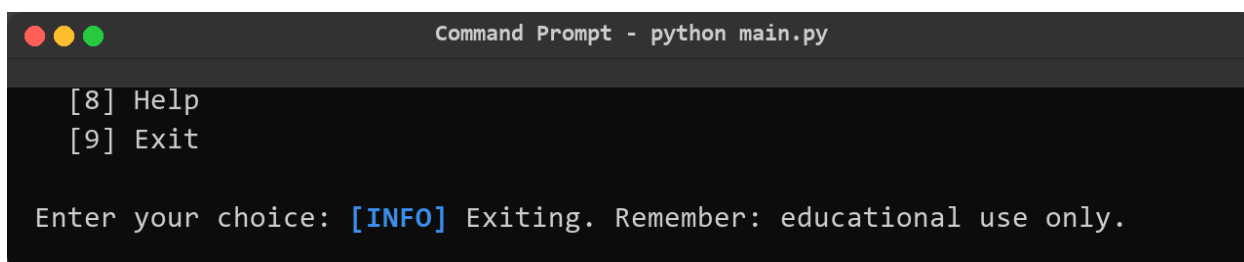


```
Command Prompt - python main.py

information, personal data, or confidential communications.
=====

MAIN MENU
-----
[1] Caesar Cipher (encrypt / decrypt)
[2] Vigenère Cipher (encrypt / decrypt)
[3] Frequency Analysis
[4] Break Caesar Cipher (automatic)
[5] Analyze Vigenère Cipher (cryptanalysis)
[6] File Encryption / Decryption
[7] Security Analysis
```

Figure 6: Start-up screen (continued).



```
Command Prompt - python main.py

[8] Help
[9] Exit

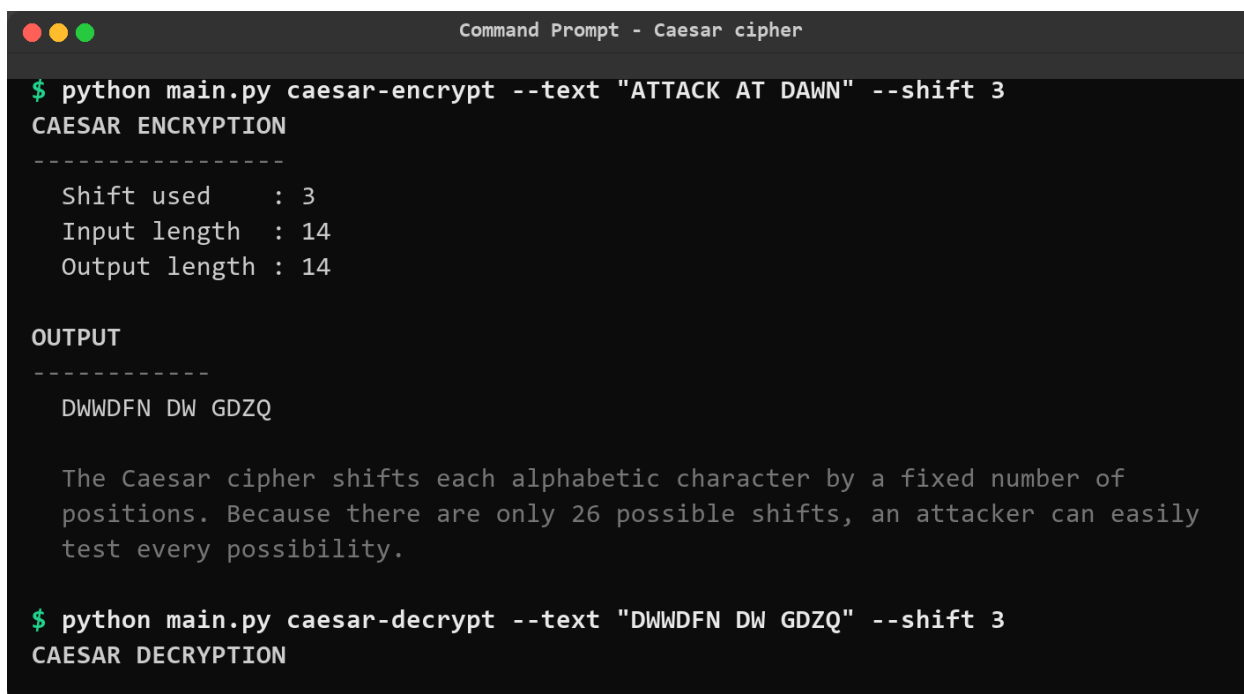
Enter your choice: [INFO] Exiting. Remember: educational use only.
```

Figure 7: Start-up screen (continued).

Step 3 – Encrypting and decrypting text with the Caesar cipher

The first functional test was a round trip through the Caesar cipher using the shift of 3 that makes the cipher famous. Encryption turns ATTACK AT DAWN into DWWDFN DW GDZQ and decryption must return the original text exactly, including the spaces, because the tool only shifts alphabetic characters. The command line form was used here so that the operation, the shift and the result are all visible on one screen.

```
python main.py caesar-encrypt --text "ATTACK AT DAWN" --shift 3
python main.py caesar-decrypt --text "DWWDFN DW GDZQ" --shift 3
```



```
Command Prompt - Caesar cipher

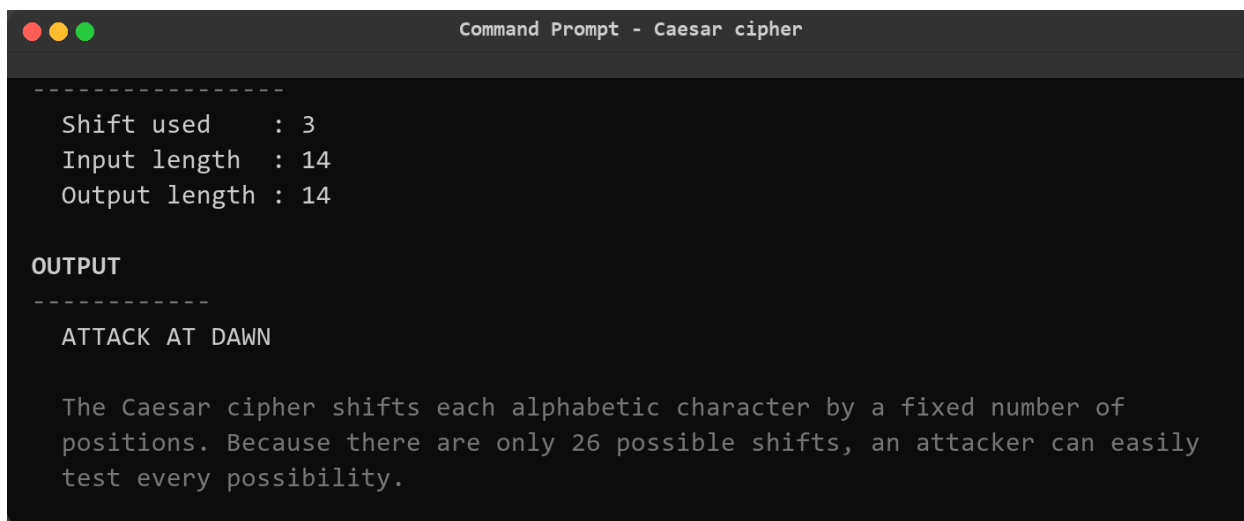
$ python main.py caesar-encrypt --text "ATTACK AT DAWN" --shift 3
CAESAR ENCRYPTION
-----
Shift used      : 3
Input length    : 14
Output length   : 14

OUTPUT
-----
DWWDFN DW GDZQ

The Caesar cipher shifts each alphabetic character by a fixed number of
positions. Because there are only 26 possible shifts, an attacker can easily
test every possibility.

$ python main.py caesar-decrypt --text "DWWDFN DW GDZQ" --shift 3
CAESAR DECRYPTION
```

Figure 8: Caesar encryption and decryption. Each run prints the shift used and the input and output lengths, and the decryption reproduces ATTACK AT DAWN exactly, confirming that the implementation is symmetric and that non-alphabetic characters are left untouched.



```
Command Prompt - Caesar cipher

-----
Shift used      : 3
Input length    : 14
Output length   : 14

OUTPUT
-----
ATTACK AT DAWN

The Caesar cipher shifts each alphabetic character by a fixed number of
positions. Because there are only 26 possible shifts, an attacker can easily
test every possibility.
```

Figure 9: Caesar encryption and decryption (continued).

Step 4 – Encrypting and decrypting text with the Vigenere cipher

The Vigenere cipher was tested next with the classic textbook example, the keyword LEMON against the plaintext ATTACKATDAWN, which is the pairing used in nearly every description of the cipher and therefore an easy way to check the implementation against an independent source. The keystream repeats every five letters, so the same plaintext letter produces different ciphertext letters at different positions – the property that defeats simple frequency analysis.

```
python main.py vigenere-encrypt --text "ATTACKATDAWN" --key LEMON
python main.py vigenere-decrypt --text "LXFOPVEFRNHR" --key LEMON
```

```
Command Prompt - Vigenere cipher

$ python main.py vigenere-encrypt --text "ATTACKATDAWN" --key LEMON
VIGENERE ENCRYPTION
-----
Key          : LEMON
Key length   : 5
Input length : 12
Output length: 12

OUTPUT
-----
LXFOPVEFRNHR

The Vigenère cipher uses a repeating keyword to apply different Caesar shifts.
Repeated-key behaviour leaks statistical patterns, which is what Kasiski
examination and Index of Coincidence analysis exploit.

$ python main.py vigenere-decrypt --text "LXFOPVEFRNHR" --key LEMON
```

Figure 10: Vigenere encryption and decryption of the standard textbook example. ATTACKATDAWN becomes LXFOPVEFRNHR with the key LEMON, and decrypting the ciphertext with the same key returns the original plaintext.

```
Command Prompt - Vigenere cipher

VIGENERE DECRYPTION
-----
Key          : LEMON
Key length   : 5
Input length : 12
Output length: 12

OUTPUT
-----
ATTACKATDAWN

The Vigenère cipher uses a repeating keyword to apply different Caesar shifts.
Repeated-key behaviour leaks statistical patterns, which is what Kasiski
examination and Index of Coincidence analysis exploit.
```

Figure 11: Vigenere encryption and decryption of the standard textbook example (continued).

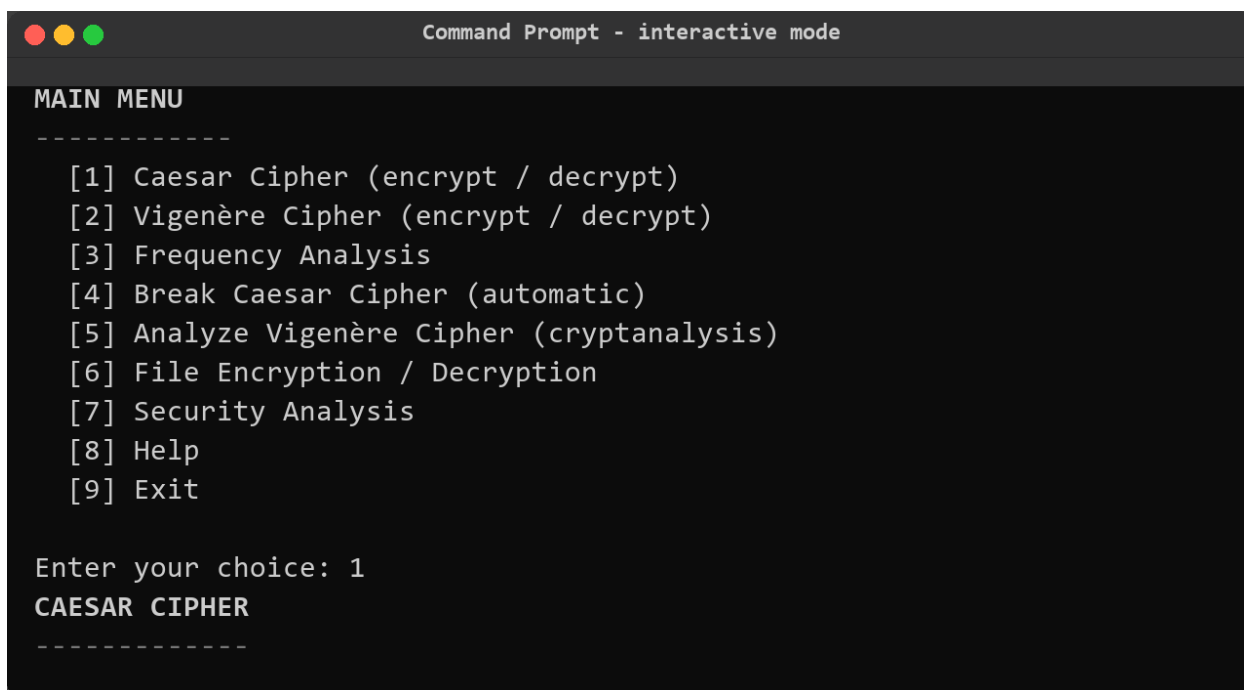
Step 5 – Running the tool in interactive mode

The same two ciphers were then exercised through the menu, which is the route a student taking their first look at the tool would normally follow. Interactive mode asks for the input source (typed text or a file), validates every answer, and offers to save the result as a report at the end of the operation. Here the text HELLO WORLD was typed, the shift 3 was accepted from the default, and the report prompt was answered with n so that no file was written.

```
python main.py

1      -> Caesar Cipher (encrypt / decrypt)
1      -> Encrypt text
```

```
1      -> Type or paste the text
HELLO WORLD
3      -> Caesar shift
n      -> do not save a report
9      -> Exit
```

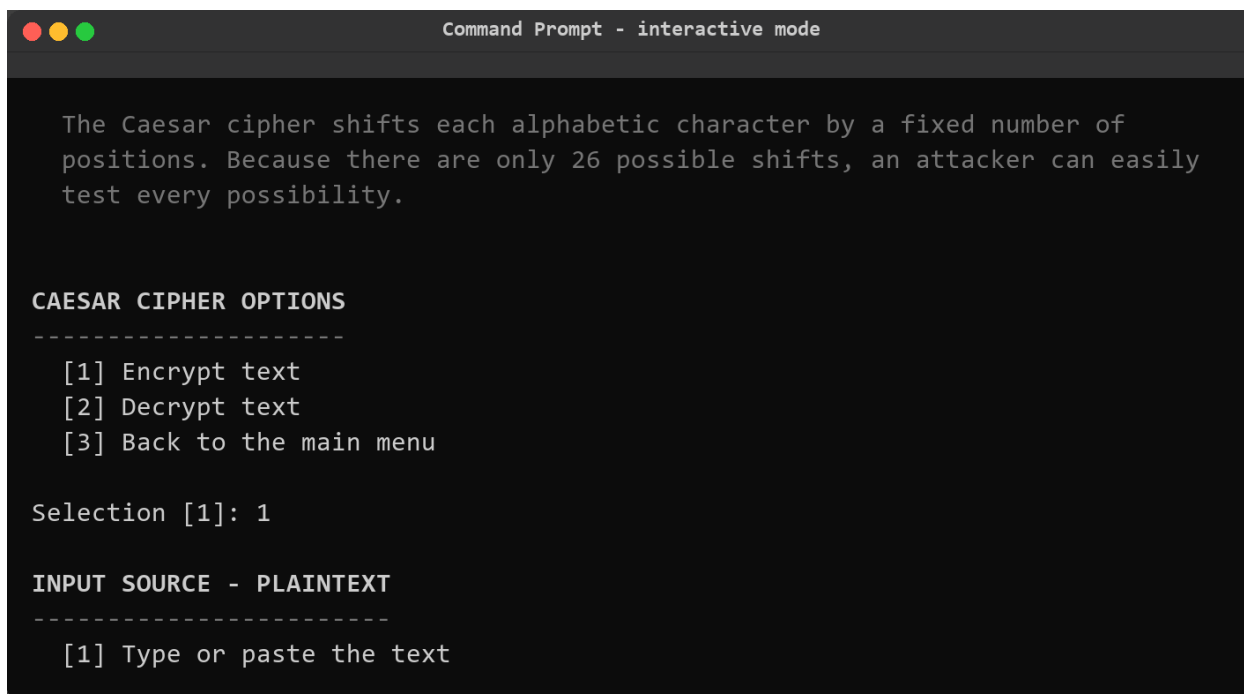


```
Command Prompt - interactive mode

MAIN MENU
-----
[1] Caesar Cipher (encrypt / decrypt)
[2] Vigenère Cipher (encrypt / decrypt)
[3] Frequency Analysis
[4] Break Caesar Cipher (automatic)
[5] Analyze Vigenère Cipher (cryptanalysis)
[6] File Encryption / Decryption
[7] Security Analysis
[8] Help
[9] Exit

Enter your choice: 1
CAESAR CIPHER
-----
```

Figure 12: Interactive walkthrough, menu and encryption. The user selects the Caesar cipher, chooses to encrypt, selects typed input, enters HELLO WORLD and the shift 3; the tool prints the shift, the input and output lengths and the ciphertext KHOOR ZRUOG.



```
Command Prompt - interactive mode

The Caesar cipher shifts each alphabetic character by a fixed number of
positions. Because there are only 26 possible shifts, an attacker can easily
test every possibility.

CAESAR CIPHER OPTIONS
-----
[1] Encrypt text
[2] Decrypt text
[3] Back to the main menu

Selection [1]: 1

INPUT SOURCE - PLAINTEXT
-----
[1] Type or paste the text
```

Figure 13: Interactive walkthrough, menu and encryption (continued).

```
Command Prompt - interactive mode

[2] Read the text from a file

Selection [1]: 1
Enter the plaintext: HELLO WORLD
Caesar shift (for example 3) [3]: 3
```

Figure 14: Interactive walkthrough, menu and encryption (continued).

```
Command Prompt - interactive mode

CAESAR ENCRYPTION
-----
Shift used      : 3
Input length   : 11
Output length  : 11

OUTPUT
-----
KHOOR ZRUOG

The Caesar cipher shifts each alphabetic character by a fixed number of
positions. Because there are only 26 possible shifts, an attacker can easily
test every possibility.
Save this analysis as a report? [y/n] [n]: n
Press Enter to return to the main menu...

MAIN MENU
```

Figure 15: Interactive walkthrough, result and exit. The educational note is repeated after the result, the tool offers to save the analysis as a report (answered n), waits for Enter and returns to the main menu, where option 9 ends the session cleanly.

```
Command Prompt - interactive mode

-----
[1] Caesar Cipher (encrypt / decrypt)
[2] Vigenère Cipher (encrypt / decrypt)
[3] Frequency Analysis
[4] Break Caesar Cipher (automatic)
[5] Analyze Vigenère Cipher (cryptanalysis)
[6] File Encryption / Decryption
[7] Security Analysis
[8] Help
[9] Exit

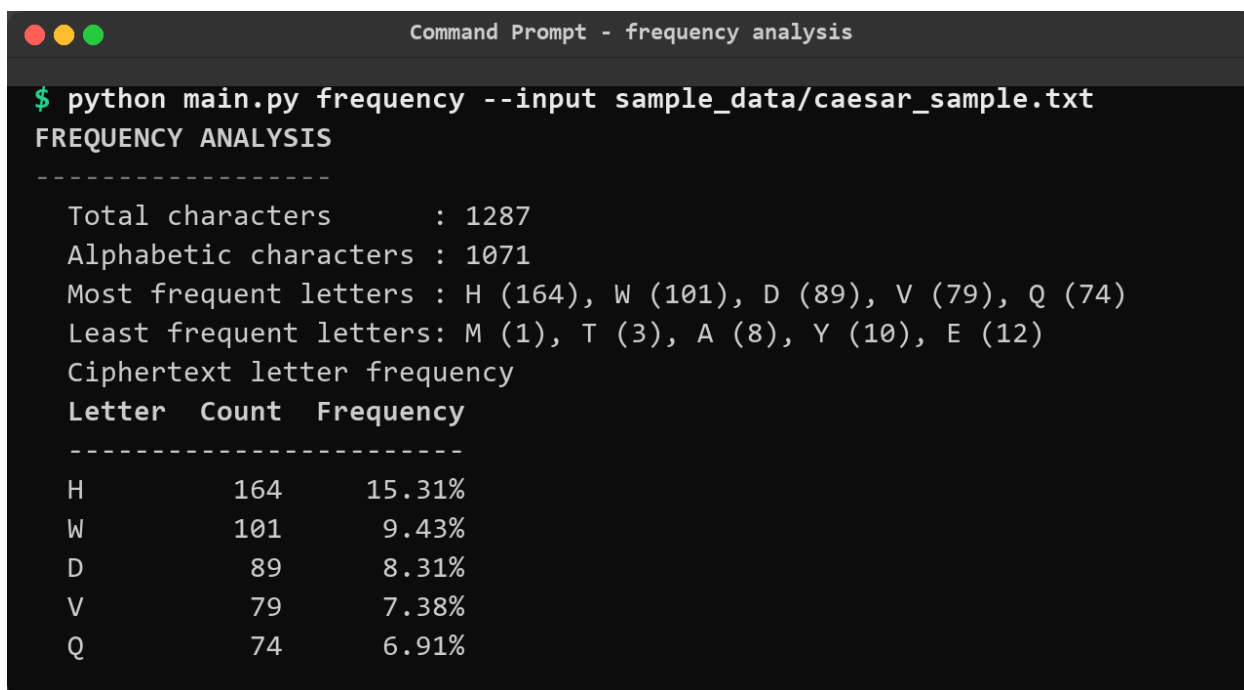
Enter your choice: 9
[INFO] End of input detected. Exiting.
```

Figure 16: Interactive walkthrough, result and exit (continued).

Step 6 – Single-letter frequency analysis

With both ciphers working, cryptanalysis began with the most basic tool. The Caesar sample file was analysed at letter level: the module counts every letter, converts the counts into percentages and prints them beside the standard English frequencies so that the distortion caused by the cipher is immediately visible. It then computes the Index of Coincidence of the whole text as supporting evidence, which is why that value also appears in this figure.

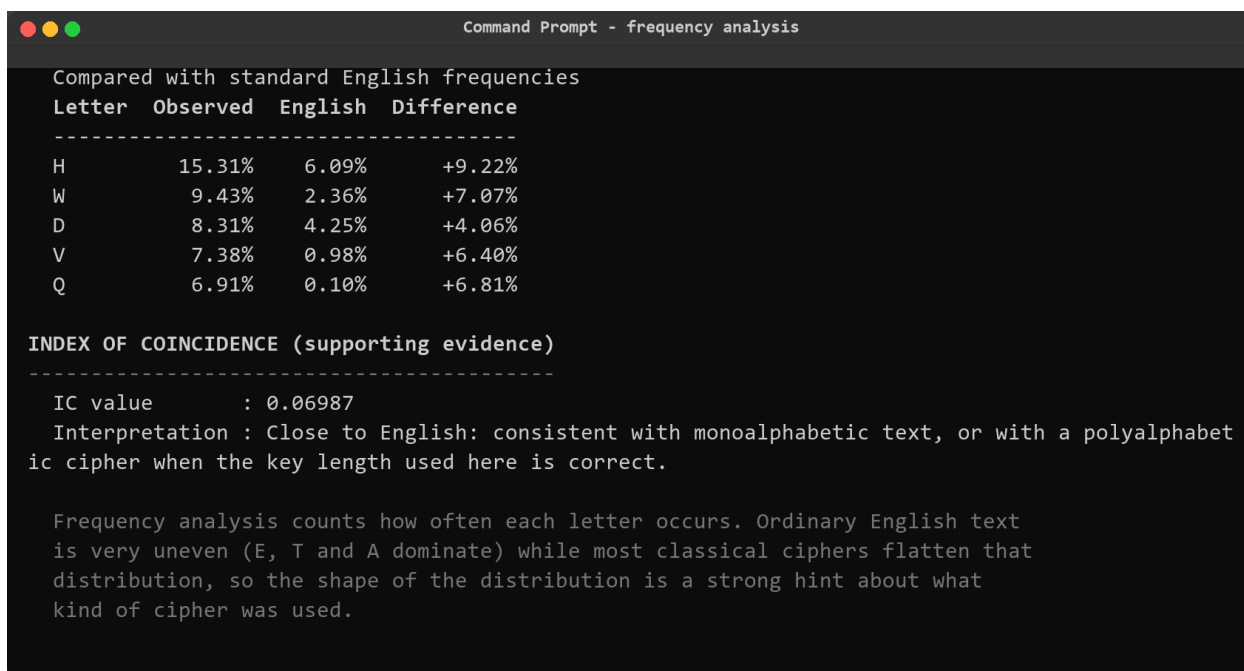
```
python main.py frequency --input sample_data/caesar_sample.txt
```



```
Command Prompt - frequency analysis

$ python main.py frequency --input sample_data/caesar_sample.txt
FREQUENCY ANALYSIS
-----
Total characters      : 1287
Alphabetic characters : 1071
Most frequent letters : H (164), W (101), D (89), V (79), Q (74)
Least frequent letters: M (1), T (3), A (8), Y (10), E (12)
Ciphertext letter frequency
Letter  Count  Frequency
-----
H       164    15.31%
W       101     9.43%
D        89     8.31%
V        79     7.38%
Q        74     6.91%
```

Figure 17: Frequency analysis of the Caesar sample. H is by far the most common letter at 15.31% against 6.09% in ordinary English while E has almost vanished, and the Index of Coincidence of 0.06987 sits close to the value for English prose – the signature of a single-alphabet cipher in which one letter of the plaintext is always replaced by one fixed letter.



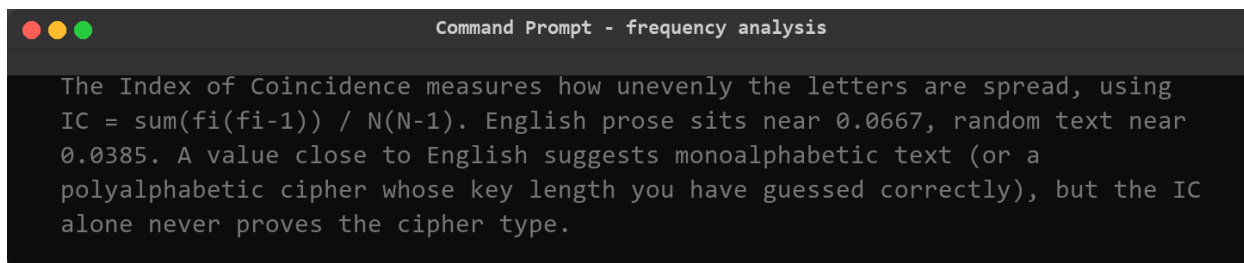
```
Command Prompt - frequency analysis

Compared with standard English frequencies
Letter  Observed  English  Difference
-----
H       15.31%   6.09%    +9.22%
W        9.43%   2.36%    +7.07%
D        8.31%   4.25%    +4.06%
V        7.38%   0.98%    +6.40%
Q        6.91%   0.10%    +6.81%

INDEX OF COINCIDENCE (supporting evidence)
-----
IC value      : 0.06987
Interpretation : Close to English: consistent with monoalphabetic text, or with a polyalphabet
ic cipher when the key length used here is correct.

Frequency analysis counts how often each letter occurs. Ordinary English text
is very uneven (E, T and A dominate) while most classical ciphers flatten that
distribution, so the shape of the distribution is a strong hint about what
kind of cipher was used.
```

Figure 18: Frequency analysis of the Caesar sample (continued).



```
Command Prompt - frequency analysis

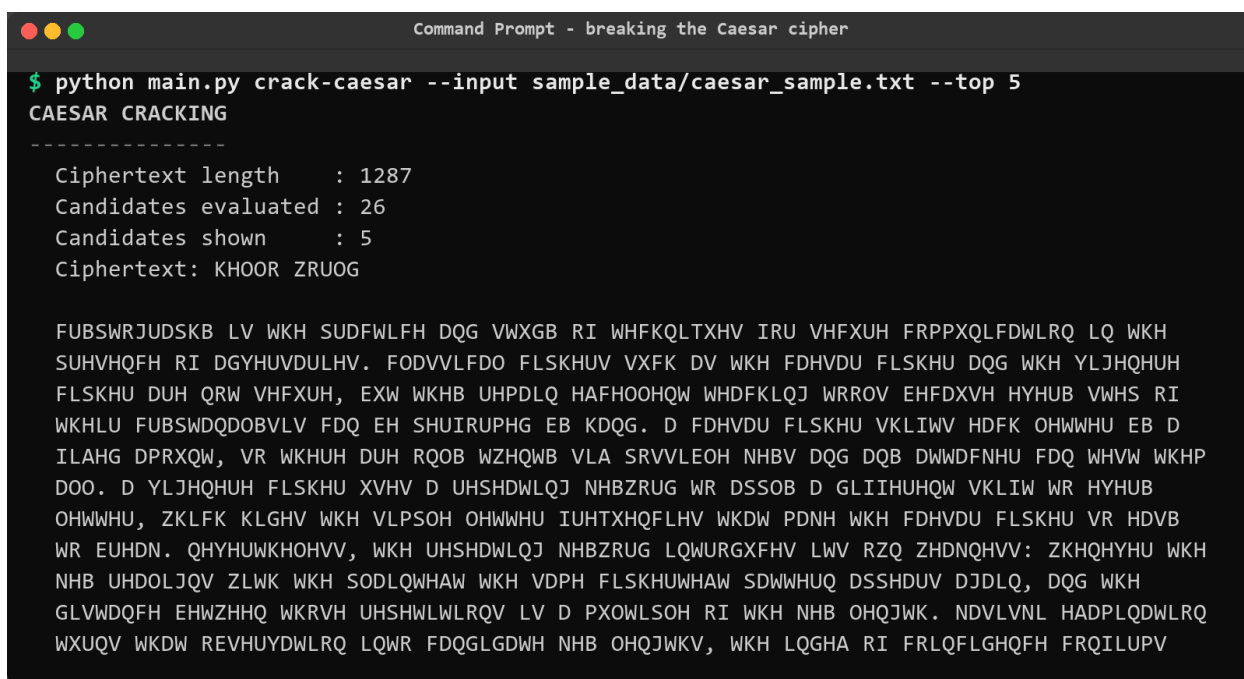
The Index of Coincidence measures how unevenly the letters are spread, using
IC = sum(fi(fi-1)) / N(N-1). English prose sits near 0.0667, random text near
0.0385. A value close to English suggests monoalphabetic text (or a
polyalphabetic cipher whose key length you have guessed correctly), but the IC
alone never proves the cipher type.
```

Figure 19: Frequency analysis of the Caesar sample (continued).

Step 7 – Breaking the Caesar cipher automatically

The distorted distribution makes the attack on the Caesar cipher almost trivial, and the cracker performs it without any hint from the user. All twenty-six possible shifts are applied to the ciphertext, each candidate is scored with the chi-square statistic against the standard English letter frequencies, and the candidates are ranked from most to least English-like. The tool then prints the best candidate together with the full decrypted text so that the result can be judged by reading it rather than by trusting a number.

```
python main.py crack-caesar --input sample_data/caesar_sample.txt --top 5
```



```
Command Prompt - breaking the Caesar cipher

$ python main.py crack-caesar --input sample_data/caesar_sample.txt --top 5
CAESAR CRACKING
-----
Ciphertext length      : 1287
Candidates evaluated   : 26
Candidates shown       : 5
Ciphertext: KHOOR ZRUOG

FUBSWRJUDSKB LV WKH SUDFWLFH DQG VWXGB RI WHFKQLTXHV IRU VHFXUH FRPPXQLFDWLRQ LQ WKH
SUHVVHQFH RI DGYHUVLULHV. FODVVLFDO FLSKHUV VXFK DV WKH FDHVDU FLSKHU DQG WKH YLJHQHUH
FLSKHU DUH QRW VHFXUH, EXW WKHB UHPDLQ HAFHOHQW WHDFKLQJ WRROV EHFDXVH HYHUB VWHS RI
WKHLU FUBSWDQDOBVLV FDQ EH SHUIRUPHG EB KDQG. D FDHVDU FLSKHU VKLIWV HDFK OHWWHU EB D
ILAHG DPRXQW, VR WKHUH DUH RQOB WZHQWB VLA SRVVLEOH NHBV DQG DQB DWWDFNHU FDQ WHVW WKHP
DOO. D YLJHQHUH FLSKHU XVHV D UHSHDWLQJ NHBZRUG WR DSSOB D GLIIHQHQW VKLIW WR HYHUB
OHWWHU, ZKLFK KLGHV WKH VLPSON OHWWHU IUHTXHQFLHV WKDW PDNH WKH FDHVDU FLSKHU VR HDVB
WR EUHDN. QHYHUWKHOVV, WKH UHSHDWLQJ NHBZRUG LQWURGXFHV LWV RZQ ZHDNQHVV: ZKHQHYHU WKH
NHB UHDOLJQV ZLWK WKH SODLQWAW WKH VDPH FLSKHUWAW SDWWHUQ DSSHUV DJDLQ, DQG WKH
GLVWDQFH EHZZHQ WKRVH UHSHWLWLRQV LV D PXOWSON RI WKH NHB OHQJWK. NDVLNL HADPLQDWLRQ
WXUQV WKDW REVHUYDWLRQ LQWR FDQLGDWH NHB OHQJWKV, WKH LQGHQFH RI FRLQFLGHQFH FRQILUPV
```

Figure 20: Automatic Caesar cracking, ranked candidates. Twenty-six candidates were evaluated; the shift of 3 scores 70.52, more than three times the second-ranked candidate at 22.94, and is the only row whose preview begins with recognisable English (HELLO WORLD, CRYPTOGRAPHY...).

```

Command Prompt - breaking the Caesar cipher

WKHP EB VKRZOLQJ WKDW HDFK FROXPQ EHKDYHV OLNH RUGLQDUB ODQJXDJH, DQG IUHTXHQFB DQDOBVLV
ILQLVKHV WKH MRE RQH FROXPQ DW D WLP. XQGHUVWDQGLQJ WKHVH DWDFNV LV WKH ILUVV VWHS
WRZDUGV XQGHUVWDQGLQJ ZKB PRGHUQ HQFUBSWLRQ XVHV ODUJH UDQGRP NHBV, DXWKHQWLFDWHG
FLSKHUWHAWV DQG FDUHIXOUB UHYLHZHG DOJRULWKPV LQVWDG RI VKDUHG VHFUHWV.
Possible solutions (ranked by English-likeness)
Rank  Shift  Score  Candidate plaintext
-----
1       3    70.52  HELLO WORLD CRYPTOGRAPHY IS THE PRACTICE AND STUDY OF TE...
2      16    22.94  URYYB JBEYQ PELCGBTENCUL VF GUR CENPGVPR NAQ FGHQL BS GR...
3      18    17.23  SPWWZ HZCWO NCJAEZRCLASJ TD ESP ACLNETNP LYO DEFOJ ZQ EP...
4      14    16.28  WTAAD LDGAS RGNEIDVGPEWN XH IWT EGPRIXRT PCS HIJSN DU IT...
5       9    15.60  BYFFI QILFX WLSJNIALUJBS CM NBY JLUWNCWY UHX MNOXS IZ NY...

```

Figure 21: Automatic Caesar cracking, ranked candidates (continued).

```

Command Prompt - recovered plaintext

BEST CANDIDATE
-----
Estimated shift : 3
Score           : 70.52
HELLO WORLD

CRYPTOGRAPHY IS THE PRACTICE AND STUDY OF TECHNIQUES FOR SECURE COMMUNICATION IN THE
PRESENCE OF ADVERSARIES. CLASSICAL CIPHERS SUCH AS THE CAESAR CIPHER AND THE VIGENERE
CIPHER ARE NOT SECURE, BUT THEY REMAIN EXCELLENT TEACHING TOOLS BECAUSE EVERY STEP OF
THEIR CRYPTANALYSIS CAN BE PERFORMED BY HAND. A CAESAR CIPHER SHIFTS EACH LETTER BY A
FIXED AMOUNT, SO THERE ARE ONLY TWENTY SIX POSSIBLE KEYS AND ANY ATTACKER CAN TEST THEM
ALL. A VIGENERE CIPHER USES A REPEATING KEYWORD TO APPLY A DIFFERENT SHIFT TO EVERY
LETTER, WHICH HIDES THE SIMPLE LETTER FREQUENCIES THAT MAKE THE CAESAR CIPHER SO EASY
TO BREAK. NEVERTHELESS, THE REPEATING KEYWORD INTRODUCES ITS OWN WEAKNESS: WHENEVER THE
KEY REALIGNS WITH THE PLAINTEXT THE SAME CIPHERTEXT PATTERN APPEARS AGAIN, AND THE
DISTANCE BETWEEN THOSE REPETITIONS IS A MULTIPLE OF THE KEY LENGTH. KASISKI EXAMINATION
TURNS THAT OBSERVATION INTO CANDIDATE KEY LENGTHS, THE INDEX OF COINCIDENCE CONFIRMS
THEM BY SHOWING THAT EACH COLUMN BEHAVES LIKE ORDINARY LANGUAGE, AND FREQUENCY ANALYSIS
FINISHES THE JOB ONE COLUMN AT A TIME. UNDERSTANDING THESE ATTACKS IS THE FIRST STEP

```

Figure 22: Automatic Caesar cracking, recovered plaintext. The best candidate is shift 3 with a score of 70.52 and the recovered plaintext matches the known text of the sample file, so no shift had to be guessed by the user. The closing notes explain that the ranking is statistical and that short or unusual messages can rank the wrong shift first.

```

Command Prompt - recovered plaintext

TOWARDS UNDERSTANDING WHY MODERN ENCRYPTION USES LARGE RANDOM KEYS, AUTHENTICATED
CIPHERTEXTS AND CAREFULLY REVIEWED ALGORITHMS INSTEAD OF SHARED SECRETS.

Automated cracking is statistical: the ranking is based on how much each
candidate looks like English. Short messages, names, technical terms or non-
English text can rank the wrong shift first, so compare the top candidates
rather than trusting the first row blindly.

Educational use only: these classical ciphers are not secure for real data.

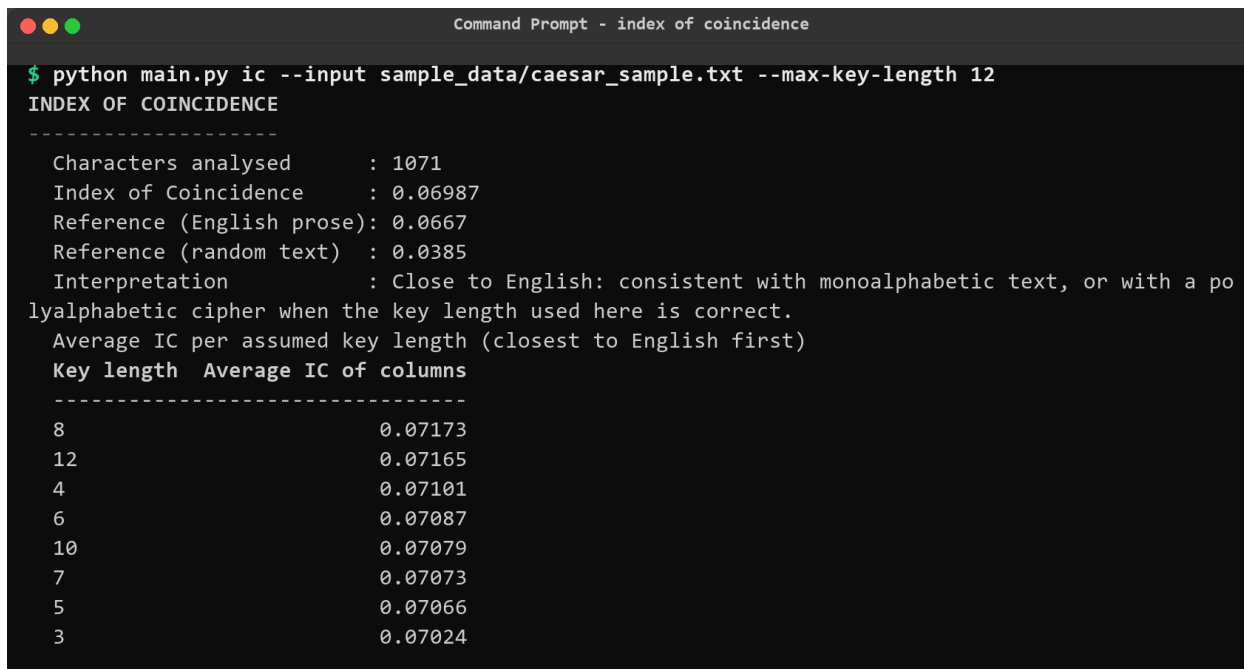
```

Figure 23: Automatic Caesar cracking, recovered plaintext (continued).

Step 8 – Measuring the Index of Coincidence

The Caesar cipher was easy precisely because every letter of the plaintext maps to one fixed letter. To show the contrast with a polyalphabetic cipher, the Index of Coincidence was measured on its own. The metric is computed as the sum over letters of $f_i(f_i-1)$ divided by $N(N-1)$; English prose sits near 0.0667 and random text near 0.0385. The same value is then recalculated for text split into two, three and up to twelve columns, because a repeating-key cipher whose key length has been guessed correctly will produce columns that each look like ordinary language and therefore raise the average.

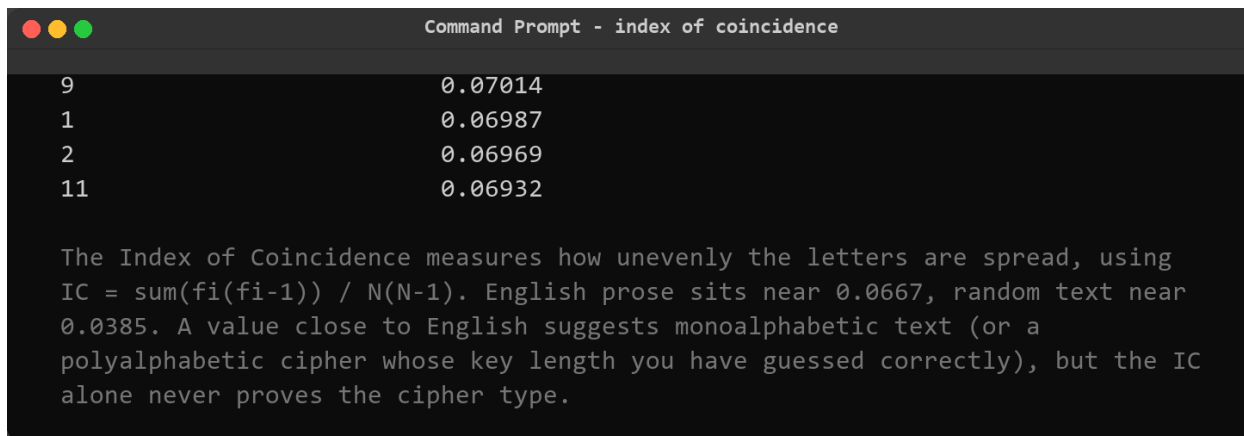
```
python main.py ic --input sample_data/caesar_sample.txt --max-key-length 12
```



```
Command Prompt - index of coincidence

$ python main.py ic --input sample_data/caesar_sample.txt --max-key-length 12
INDEX OF COINCIDENCE
-----
Characters analysed      : 1071
Index of Coincidence     : 0.06987
Reference (English prose): 0.0667
Reference (random text)  : 0.0385
Interpretation           : Close to English: consistent with monoalphabetic text, or with a polyalphabetic cipher when the key length used here is correct.
Average IC per assumed key length (closest to English first)
Key length  Average IC of columns
-----
8           0.07173
12          0.07165
4           0.07101
6           0.07087
10          0.07079
7           0.07073
5           0.07066
3           0.07024
```

Figure 24: Index of Coincidence of the Caesar sample. The overall value of 0.06987 is above the 0.0667 reference for English, and because splitting a single-alphabet cipher into columns cannot increase the value, every key length stays close to the same level – there is no key length that stands out, exactly as expected for a monoalphabetic cipher.



```
Command Prompt - index of coincidence

9           0.07014
1           0.06987
2           0.06969
11          0.06932

The Index of Coincidence measures how unevenly the letters are spread, using
IC = sum(fi(fi-1)) / N(N-1). English prose sits near 0.0667, random text near
0.0385. A value close to English suggests monoalphabetic text (or a
polyalphabetic cipher whose key length you have guessed correctly), but the IC
alone never proves the cipher type.
```

Figure 25: Index of Coincidence of the Caesar sample (continued).

Step 9 – Kasiski examination of the Vigenere sample

Attention then moved to the Vigenere sample, which had been produced with the keyword LEMON. Kasiski examination searches the ciphertext for fragments that occur more than once. In a repeating-key cipher an exact repeat can only happen when the key realigns with the plaintext, so the distance between two occurrences of the same fragment must be a multiple of the key length. The tool lists

those repeated sequences, factorises the distances, ranks the factors by how much evidence supports them and finally prints a table of candidate key lengths.

```
python main.py kasiski --input sample_data/vigenere_sample.txt
```

```

Command Prompt - Kasiski examination

$ python main.py kasiski --input sample_data/vigenere_sample.txt
KASISKI EXAMINATION
-----
Characters analysed           : 1072
Sequence length searched     : 3-5
Repeated sequences found     : 50

Repeated sequences (positions are 0-based indices into the letters):
OWCSI x3 positions: 102, 602, 997
      distances: 500, 395 -> factors: 5, 2, 4, 10, 20, 25
WCSID x3 positions: 103, 603, 998
      distances: 500, 395 -> factors: 5, 2, 4, 10, 20, 25
CSIDH x2 positions: 604, 999
      distances: 395 -> factors: 5
CXSAR x2 positions: 686, 746

```

Figure 26: Kasiski examination, repeated sequences. Fifty repeated sequences were found; the five-letter sequences OWCSI and WCSID each occur three times at distances of 500 and 395 apart, and every one of those distances is divisible by 5, which is the strongest possible hint that the key length is 5. The tool also warns in writing that these are candidate lengths and not facts.

```

Command Prompt - Kasiski examination

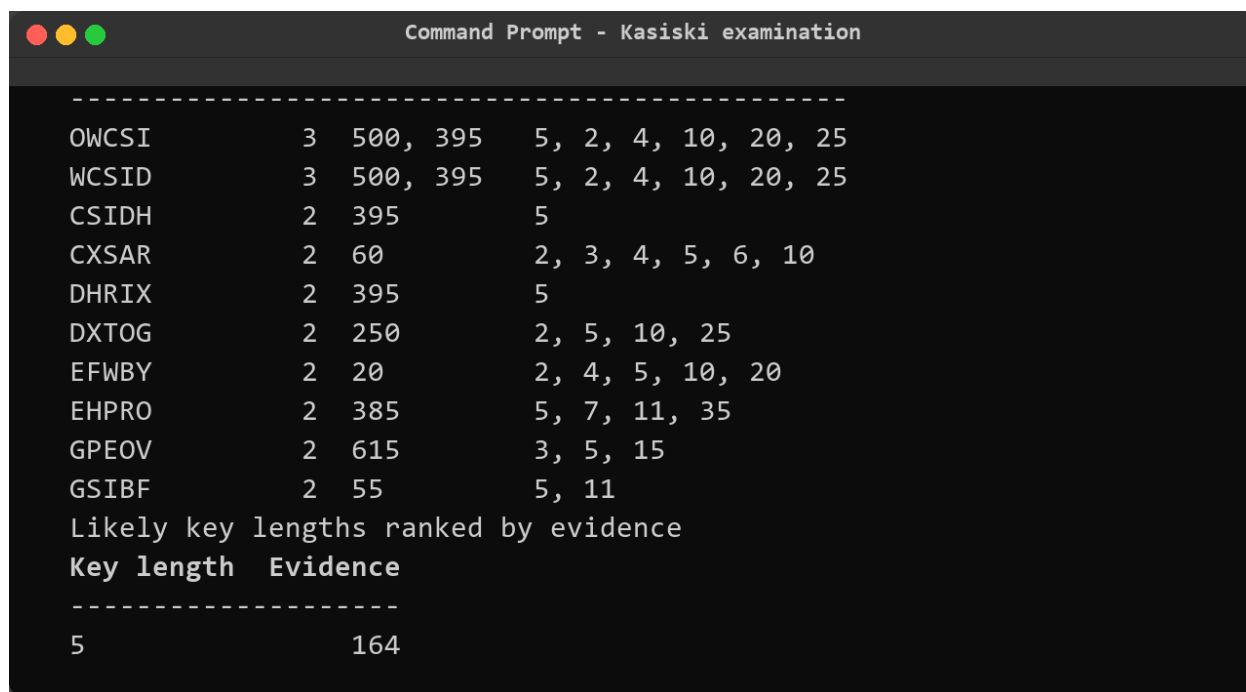
distances: 60 -> factors: 2, 3, 4, 5, 6, 10, 12, 15
DHRIX x2 positions: 607, 1002
      distances: 395 -> factors: 5
DXTOG x2 positions: 465, 715
      distances: 250 -> factors: 2, 5, 10, 25
EFWBY x2 positions: 706, 726
      distances: 20 -> factors: 2, 4, 5, 10, 20
EHPRO x2 positions: 458, 843
      distances: 385 -> factors: 5, 7, 11, 35
... and 42 more

Most common distance factors : 5 (61 hits), 2 (30 hits), 10 (30 hits), 4 (25 hits), 20 (25 hit
s), 25 (15 hits), 3 (12 hits), 15 (12 hits), 6 (10 hits), 12 (10 hits)
Likely key lengths           : 5, 2, 4, 10, 3, 6, 20, 7, 15, 12

Reminder: these are candidate key lengths derived from repeated sequences. Kasiski examination
does not prove the key length and does not recover the key.
Repeated sequences
Sequence Count Distances Shared factors

```

Figure 27: Kasiski examination, repeated sequences (continued).

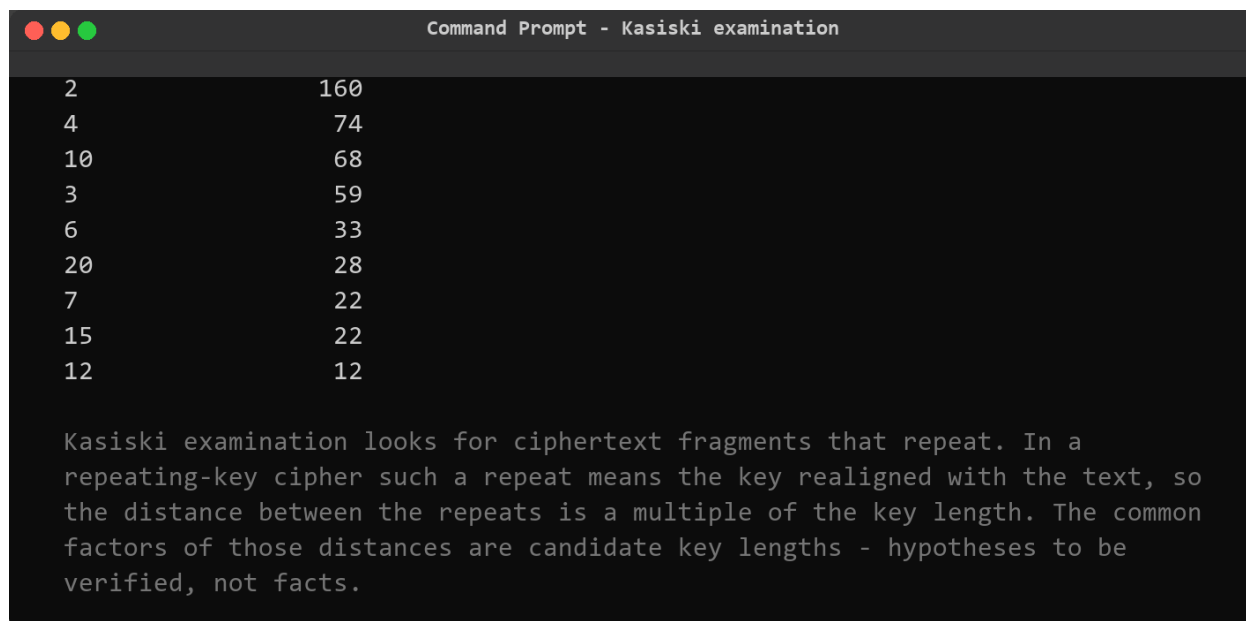


```

-----
OWCSI      3  500, 395    5, 2, 4, 10, 20, 25
WCSID      3  500, 395    5, 2, 4, 10, 20, 25
CSIDH      2   395         5
CXSAR      2   60        2, 3, 4, 5, 6, 10
DHRIX      2   395         5
DXTOG      2  250        2, 5, 10, 25
EFWBY      2   20        2, 4, 5, 10, 20
EHPRO      2  385        5, 7, 11, 35
GPEOV      2  615        3, 5, 15
GSIBF      2   55        5, 11
Likely key lengths ranked by evidence
Key length  Evidence
-----
5              164

```

Figure 28: Kasiski examination, distances and factors. The repeated sequences are tabulated with their distances and shared factors, and the ranked list of likely key lengths places length 5 first with 164 points of evidence, ahead of length 2 (160) and length 4 (74).



```

2          160
4          74
10         68
3          59
6          33
20         28
7          22
15         22
12         12

Kasiski examination looks for ciphertext fragments that repeat. In a
repeating-key cipher such a repeat means the key realigned with the text, so
the distance between the repeats is a multiple of the key length. The common
factors of those distances are candidate key lengths - hypotheses to be
verified, not facts.

```

Figure 29: Kasiski examination, distances and factors (continued).

Step 10 – Full Vigenere cryptanalysis and key recovery

Kasiski examination alone can suggest a wrong length when the text is short, so the tool combines it with the Index of Coincidence in a single workflow. It computes the overall IC of the ciphertext, calculates the average IC of the columns for every candidate key length, adds the Kasiski evidence and produces one combined score. The winning length is then used to split the ciphertext into that many columns, and each column is solved as an independent Caesar cipher: the three most likely letters for every key position are suggested, complete candidate keys are built from those suggestions, and each candidate is scored by how much its decryption looks like English.

```
python main.py analyze-vigenere --input sample_data/vigenere_sample.txt
```

```

Command Prompt - Vigenere cryptanalysis

$ python main.py analyze-vigenere --input sample_data/vigenere_sample.txt
VIGENERE CRYPTANALYSIS
-----
Characters analysed           : 1072
Index of Coincidence         : 0.04351
IC interpretation            : Low: consistent with polyalphabetic or otherwise randomised text; letter frequencies have been flattened.
Repeated sequences found     : 50

Candidate key lengths (combined evidence, estimated):
Key length  Avg IC      Kasiski hits  Score
-----
          5    0.07079         164    0.948
         10    0.07039          68    0.865
         15    0.06955          22    0.834
         20    0.07200          28    0.808
          2    0.04341         160    0.699

Reminder: key lengths above are estimates from statistics, not facts.
Repeated sequences found by Kasiski examination

```

Figure 30: Vigenere cryptanalysis, key length estimate. The overall Index of Coincidence of 0.04351 is close to the 0.0385 expected from randomised text, which is the signature of a polyalphabetic cipher. The combined ranking nevertheless puts key length 5 first because its column IC of 0.07079 is English-like and the Kasiski evidence is the strongest, even though length 2 collects almost as many Kasiski hits.

```

Command Prompt - Vigenere cryptanalysis

Sequence  Count  Distances  Shared factors
-----
OWCSI      3  500, 395  5, 2, 4, 10, 20, 25
WCSID      3  500, 395  5, 2, 4, 10, 20, 25
CSIDH      2   395    5
CXSAR      2   60     2, 3, 4, 5, 6, 10
DHRIX      2   395    5
DXTOG      2  250     2, 5, 10, 25
EFWBY      2   20     2, 4, 5, 10, 20
EHPRO      2  385     5, 7, 11, 35
GPEOV      2  615     3, 5, 15

```

Figure 31: Vigenere cryptanalysis, key length estimate (continued).

```

Command Prompt - key recovery

GSIBF      2  55      5, 11
Candidate key lengths (estimated)
Key length  Average IC  Kasiski hits  Combined score
-----
5           0.07079      164           0.948
10          0.07039       68           0.865
15          0.06955       22           0.834
20          0.07200       28           0.808
2           0.04341      160           0.699

KEY RECOVERY ASSUMING KEY LENGTH 5 (estimated)
-----
Per-column key letter candidates
Key position  Suggested letters (score)
-----

```

Figure 32: Vigenere cryptanalysis, per column key recovery. Assuming key length 5, each of the five key positions is solved on its own and the suggested letters are L, E, M, O and N with fitness scores between 50 and 58 against alternatives in the low twenties. The assembled candidate keys are then ranked, and LEMON is first with a score of 70.15.

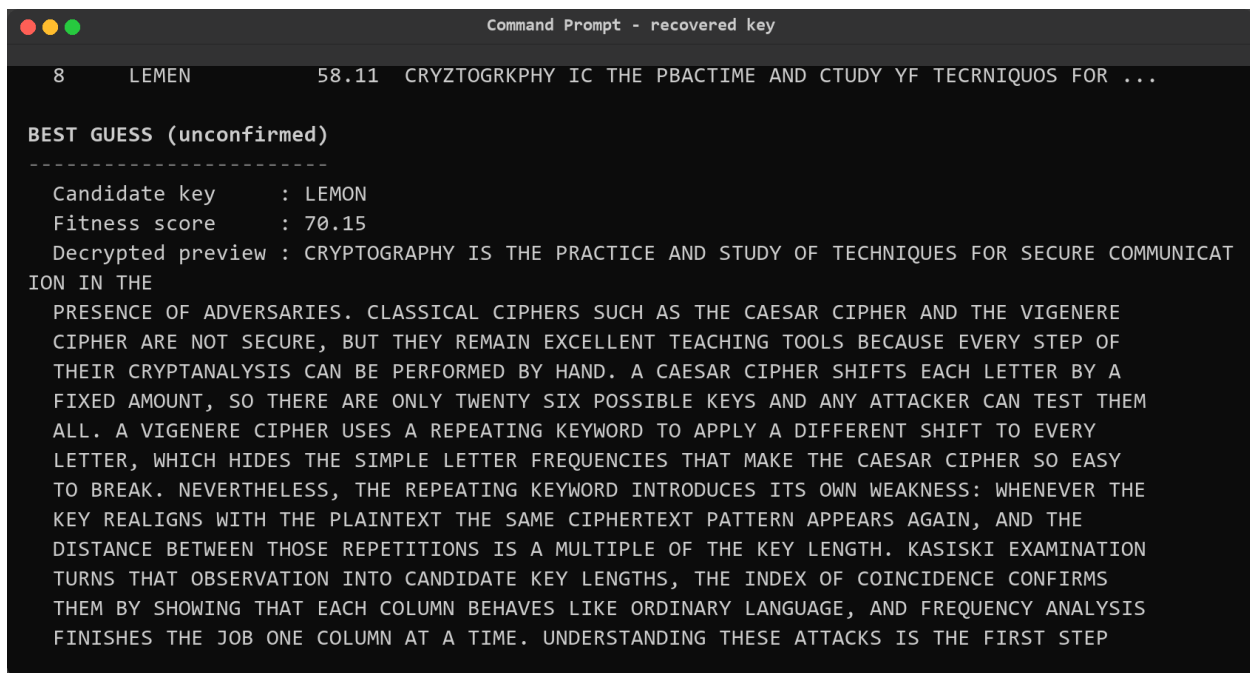
```

Command Prompt - key recovery

1      L (57.31), Y (22.33), W (18.99)
2      E (56.59), P (23.98), Q (22.44)
3      M (57.94), B (19.98), Z (18.26)
4      O (50.87), E (18.41), D (18.09)
5      N (58.43), A (19.46), C (18.00)
Candidate keys and their decryptions (estimated)
Rank  Candidate key  Score  Decrypted preview
-----
1     LEMON          70.15  CRYPTOGRAPHY IS THE PRACTICE AND STUDY OF TECHNIQUES FOR ...
2     LQMON          61.99  CFYPTOURAPHM IS THS PRACHICE ABD STURY OF TSCHNIEUES FCR ...
3     YEMON          60.24  PRYPTBGRAPUY IS TUE PRAPTICE NND STHDY OF GECHNVQUES SOR ...
4     LEMDN          59.41  CRYATOGRLPHY ID THE PRACTINE AND DTUDY ZF TECSNIQUPS FOR ...
5     LEBON          59.28  CRJPTOGCAPHY TS THE ARACTTCE ANO STUDJ OF TENHNIQFES FOC ...
6     LEMOA          59.15  CRYPGOGRACHY IS GHE PRNCTICR AND SGUDY OS TECHAIQUEF FOR ...
7     LPMON          59.04  CGYPTOVRAPHN IS THT PRACIICE ACD STUSY OF TTCHNIFUES FDR ...

```

Figure 33: Vigenere cryptanalysis, per column key recovery (continued).

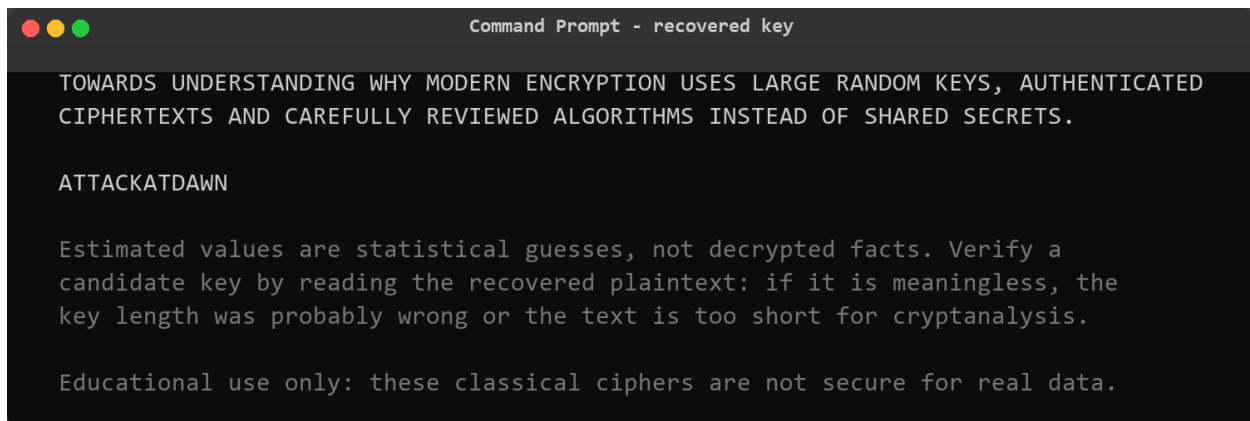


```
Command Prompt - recovered key

8 LEMON 58.11 CRYZTOGRKPHY IC THE PBACTIME AND CTUDY YF TECRNIQUOS FOR ...

BEST GUESS (unconfirmed)
-----
Candidate key      : LEMON
Fitness score      : 70.15
Decrypted preview  : CRYPTOGRAPHY IS THE PRACTICE AND STUDY OF TECHNIQUES FOR SECURE COMMUNICAT
ION IN THE
PRESENCE OF ADVERSARIES. CLASSICAL CIPHERS SUCH AS THE CAESAR CIPHER AND THE VIGENERE
CIPHER ARE NOT SECURE, BUT THEY REMAIN EXCELLENT TEACHING TOOLS BECAUSE EVERY STEP OF
THEIR CRYPTANALYSIS CAN BE PERFORMED BY HAND. A CAESAR CIPHER SHIFTS EACH LETTER BY A
FIXED AMOUNT, SO THERE ARE ONLY TWENTY SIX POSSIBLE KEYS AND ANY ATTACKER CAN TEST THEM
ALL. A VIGENERE CIPHER USES A REPEATING KEYWORD TO APPLY A DIFFERENT SHIFT TO EVERY
LETTER, WHICH HIDES THE SIMPLE LETTER FREQUENCIES THAT MAKE THE CAESAR CIPHER SO EASY
TO BREAK. NEVERTHELESS, THE REPEATING KEYWORD INTRODUCES ITS OWN WEAKNESS: WHENEVER THE
KEY REALIGNS WITH THE PLAINTEXT THE SAME CIPHERTEXT PATTERN APPEARS AGAIN, AND THE
DISTANCE BETWEEN THOSE REPETITIONS IS A MULTIPLE OF THE KEY LENGTH. KASISKI EXAMINATION
TURNS THAT OBSERVATION INTO CANDIDATE KEY LENGTHS, THE INDEX OF COINCIDENCE CONFIRMS
THEM BY SHOWING THAT EACH COLUMN BEHAVES LIKE ORDINARY LANGUAGE, AND FREQUENCY ANALYSIS
FINISHES THE JOB ONE COLUMN AT A TIME. UNDERSTANDING THESE ATTACKS IS THE FIRST STEP
```

Figure 34: Vigenere cryptanalysis, recovered keyword. The best guess is the key LEMON with a fitness score of 70.15, and its decryption is readable English from the first word. The tool still labels the result unconfirmed and reminds the user to verify a candidate key by reading the recovered plaintext, which is what makes the difference between an estimate and a result.



```
Command Prompt - recovered key

TOWARDS UNDERSTANDING WHY MODERN ENCRYPTION USES LARGE RANDOM KEYS, AUTHENTICATED
CIPHERTEXTS AND CAREFULLY REVIEWED ALGORITHMS INSTEAD OF SHARED SECRETS.

ATTACKATDAWN

Estimated values are statistical guesses, not decrypted facts. Verify a
candidate key by reading the recovered plaintext: if it is meaningless, the
key length was probably wrong or the text is too short for cryptanalysis.

Educational use only: these classical ciphers are not secure for real data.
```

Figure 35: Vigenere cryptanalysis, recovered keyword (continued).

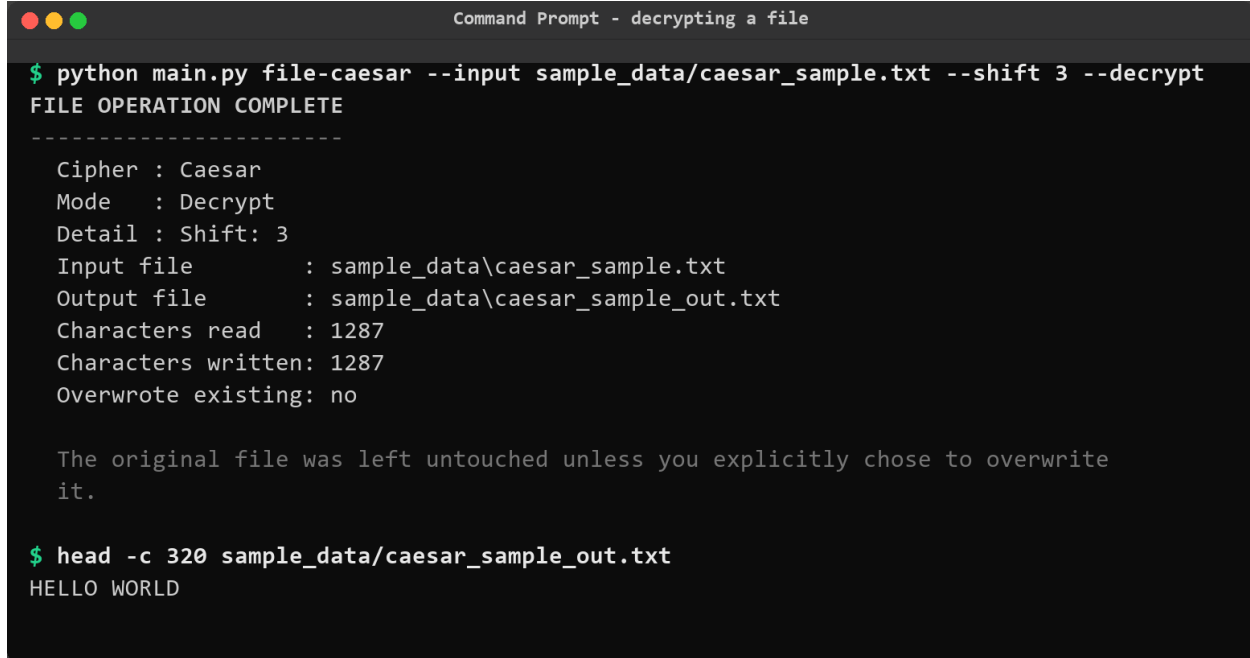
The recovered keyword LEMON is correct – it is the key that was used to build the sample file – and it was obtained without any knowledge of the plaintext, using nothing but the ciphertext and three statistical techniques applied in the right order. This is the central demonstration of the project.

Step 11 – Encrypting and decrypting whole files

Real work happens on files rather than on typed strings, so the same ciphers were applied to the sample files on disk. The file handler reads the input, transforms the text and writes the result to a new file without ever modifying the source: the default output name adds the suffix `_out`, and replacing an existing file requires the explicit `--overwrite` flag. The first file, held as Caesar ciphertext, was decrypted back into plain English; the resulting plaintext was then re-encrypted with the Vigenere cipher and a different key to produce a second ciphertext for comparison.

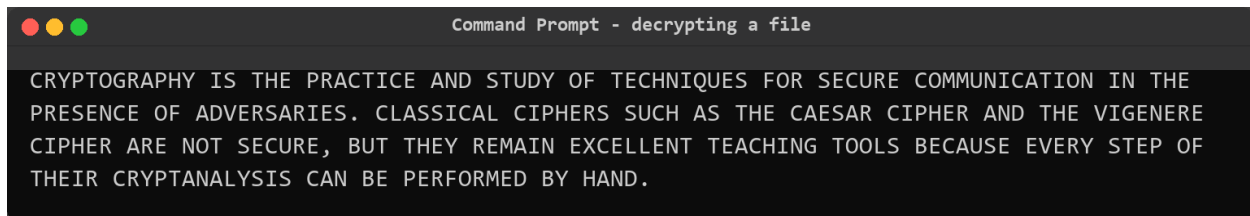
```
python main.py file-caesar --input sample_data/caesar_sample.txt --shift 3 --
decrypt
python main.py file-vigenere --input sample_data/caesar_sample_out.txt --key
```

```
LEMON \  
  --output sample_data/message_vigenere.txt
```



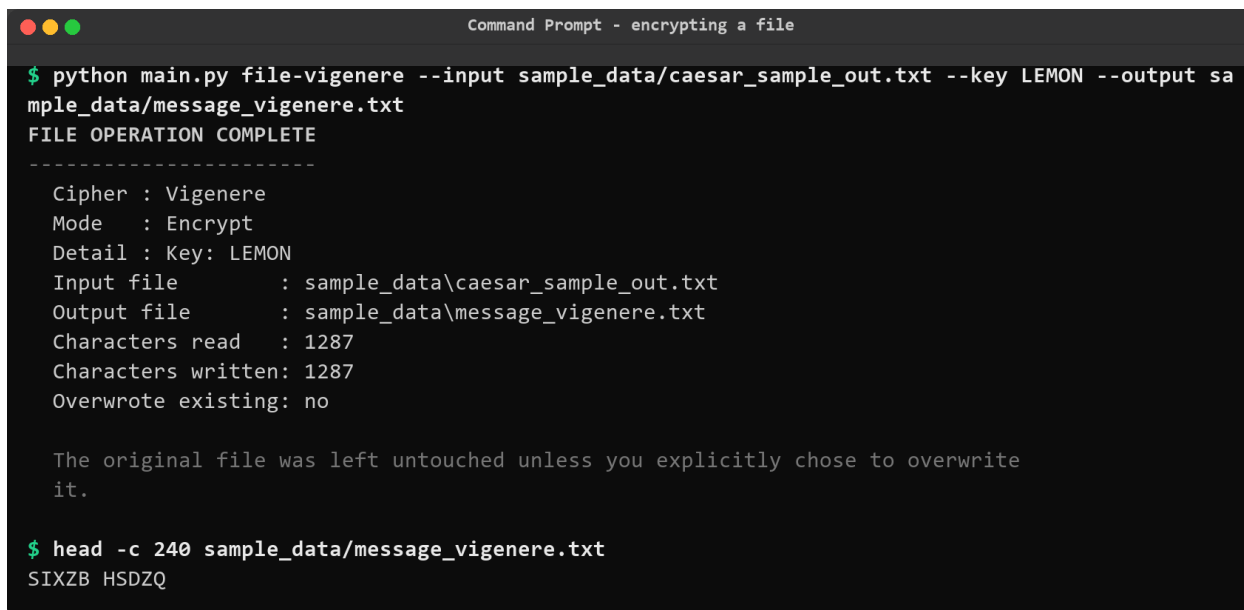
```
Command Prompt - decrypting a file  
  
$ python main.py file-caesar --input sample_data/caesar_sample.txt --shift 3 --decrypt  
FILE OPERATION COMPLETE  
-----  
Cipher : Caesar  
Mode   : Decrypt  
Detail : Shift: 3  
Input file      : sample_data\caesar_sample.txt  
Output file     : sample_data\caesar_sample_out.txt  
Characters read  : 1287  
Characters written: 1287  
Overwrote existing: no  
  
The original file was left untouched unless you explicitly chose to overwrite it.  
  
$ head -c 320 sample_data/caesar_sample_out.txt  
HELLO WORLD
```

Figure 36: Decrypting a file with the Caesar cipher. 1287 characters were read and 1287 written to `caesar_sample_out.txt`, the tool confirms that no existing file was overwritten, and the first 320 characters printed from the output file are readable English beginning with `HELLO WORLD`.



```
Command Prompt - decrypting a file  
  
CRYPTOGRAPHY IS THE PRACTICE AND STUDY OF TECHNIQUES FOR SECURE COMMUNICATION IN THE  
PRESENCE OF ADVERSARIES. CLASSICAL CIPHERS SUCH AS THE CAESAR CIPHER AND THE VIGENERE  
CIPHER ARE NOT SECURE, BUT THEY REMAIN EXCELLENT TEACHING TOOLS BECAUSE EVERY STEP OF  
THEIR CRYPTANALYSIS CAN BE PERFORMED BY HAND.
```

Figure 37: Decrypting a file with the Caesar cipher (continued).



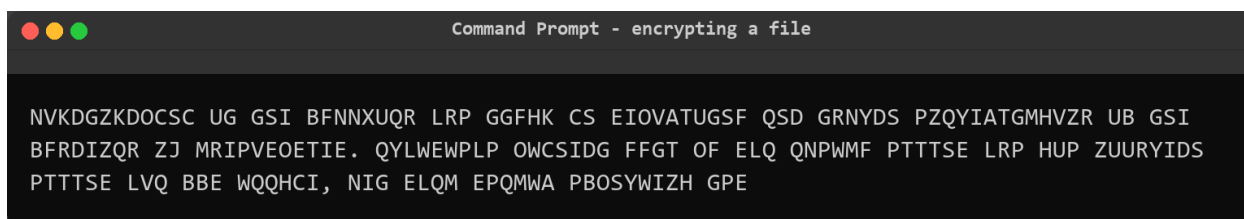
```
Command Prompt - encrypting a file

$ python main.py file-vigenere --input sample_data/caesar_sample_out.txt --key LEMON --output sample_data/message_vigenere.txt
FILE OPERATION COMPLETE
-----
Cipher : Vigenere
Mode   : Encrypt
Detail : Key: LEMON
Input file      : sample_data\caesar_sample_out.txt
Output file     : sample_data\message_vigenere.txt
Characters read  : 1287
Characters written: 1287
Overwrote existing: no

The original file was left untouched unless you explicitly chose to overwrite it.

$ head -c 240 sample_data/message_vigenere.txt
SIXZB HSDZQ
```

Figure 38: Encrypting a file with the Vigenere cipher. The decrypted file was re-encrypted with the key LEMON into message_vigenere.txt, and the preview shows that the identical plaintext now starts with SIXZB HSDZQ instead of KHOOR ZRUOG, confirming that the two ciphers transform the same bytes differently.



```
Command Prompt - encrypting a file

NVKDGZKDOCSC UG GSI BFNNXUQR LRP GGFHK CS EIOVATUGSF QSD GRNYDS PZQYIATGMHVZR UB GSI
BFRDIZQR ZJ MRIPVEOETIE. QYLWEWPLP OWCSIDG FFGT OF ELQ QNPWMF PTTTSE LRP HUP ZUURYIDS
PTTTSE LVQ BBE WQQHCI, NIG ELQM EPQMWA PBOSYWIZH GPE
```

Figure 39: Encrypting a file with the Vigenere cipher (continued).

Step 12 – Assessing the strength of both ciphers

Finally the tool was asked to state, in plain language, how much protection each cipher actually offers. The security module reports the keyspace, the strengths, the weaknesses and the known cryptanalytic attacks for the chosen cipher, and finishes with the practical verdict. Both ciphers were assessed so that the two reports can be compared.

```
python main.py security --cipher caesar
python main.py security --cipher vigenere
```

```
Command Prompt - security assessment

$ python main.py security --cipher caesar
SECURITY / STRENGTH ANALYSIS
-----

CAESAR CIPHER
-----

Cipher       : Caesar Cipher
Security rating : VERY WEAK
Keyspace     : 25 effective keys (26 shifts, shift 0 leaves the text unchanged)

Strengths:
- Trivial to implement and to explain, which makes it a good teaching tool.
- Encryption and decryption are fast and need no key exchange infrastructure.
- Useful as the building block that the Vigenere cipher generalises.

Weaknesses:
- Extremely small keyspace: the whole key can be tested in milliseconds.
- Easily broken with single-letter frequency analysis.
```

Figure 40: Security assessment of the Caesar cipher. It is rated VERY WEAK with an effective keyspace of only 25 keys; the report lists the small keyspace, vulnerability to single-letter frequency analysis, the surviving letter patterns and the absence of authentication as weaknesses, and ends with the verdict never to use it for real information.

```
Command Prompt - security assessment

- The shift is preserved by the alphabet, so letter patterns survive intact.
- Identical plaintext always produces identical ciphertext (no randomness).
- No authentication: ciphertext can be modified without detection.

Known cryptanalytic attacks:
- Exhaustive brute force over all 26 shifts (see the 'Break Caesar Cipher' option).
- Chi-square / frequency scoring to rank candidate plaintexts automatically.
- Known-plaintext attack: one known letter reveals the entire shift.

Modern use       : Legacy puzzle games, CTF warm-up challenges and cryptography education only.
Verdict         : Never use the Caesar cipher to protect real information.
=====
EDUCATIONAL USE ONLY

This project demonstrates classical cryptography and cryptanalysis.
Caesar and Vigenere ciphers are historically important but are NOT
secure modern encryption algorithms.
```

Figure 41: Security assessment of the Caesar cipher (continued).

```
Command Prompt - security assessment

Do not use this project to protect passwords, credentials, financial
information, personal data, or confidential communications.
=====
```

Figure 42: Security assessment of the Caesar cipher (continued).

```
Command Prompt - security assessment

$ python main.py security --cipher vigenere
SECURITY / STRENGTH ANALYSIS
-----

VIGENERE CIPHER
-----

Cipher      : Vigenere Cipher
Security rating : WEAK / EDUCATIONAL ONLY
Keyspace    : 26^L keys for a key of length L (L=8 gives about 2.1e11 keys), but the keysp
ace is not the weakness

Strengths:
- Much larger keyspace than the Caesar cipher; historically called 'le chiffre indechiffraabl
e'.
- Defeats naive single-letter frequency analysis when the key is as long as the message.
- Introduces the polyalphabetic idea that underlies the one-time pad.

Weaknesses:
- A repeating key means the same shift is reused, which leaks structure.
- Kasiski examination recovers probable key lengths from repeated sequences.
```

Figure 43: Security assessment of the Vigenere cipher. It is rated WEAK / EDUCATIONAL ONLY. The report explains the key insight of this project: the keyspace of 26^L for a key of length L is large, but it is not the weakness – the repeating keyword leaks the key length, and once the length is known the cipher decomposes into L independent Caesar ciphers.

```
Command Prompt - security assessment

- Index of Coincidence per column reveals the key length and enables per-column analysis.
- Once the key length is known, each column is an independent Caesar cipher.
- Key length is effectively limited by human-memorable words, so real keys are short.
- No diffusion, no authentication and no randomness.

Known cryptanalytic attacks:
- Kasiski examination: repeated sequences give distances whose factors suggest the key lengt
h.
- Index of Coincidence: the average IC of the columns peaks at the true key length.
- Per-column frequency analysis (chi-square) guesses each key letter independently.
- Friedman / mutual Index of Coincidence estimates the key length statistically.

Modern use      : Cryptography education, puzzle solving and historical study only.
Verdict        : Suitable for teaching polyalphabetic ciphers, never for sensitive data.
=====
EDUCATIONAL USE ONLY

This project demonstrates classical cryptography and cryptanalysis.
Caesar and Vigenere ciphers are historically important but are NOT
secure modern encryption algorithms.
```

Figure 44: Security assessment of the Vigenere cipher (continued).

```
Command Prompt - security assessment

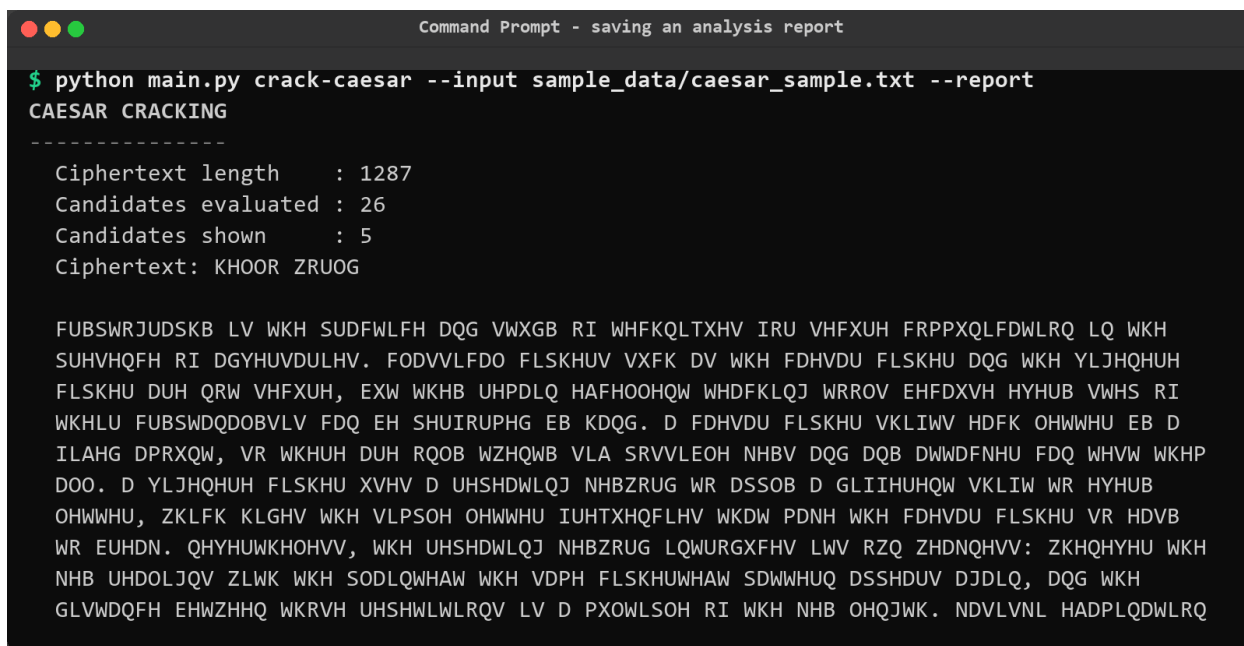
Do not use this project to protect passwords, credentials, financial
information, personal data, or confidential communications.
=====
```

Figure 45: Security assessment of the Vigenere cipher (continued).

Step 13 – Saving an analysis report to disk

Every operation can write its complete result to a timestamped text file under reports/ by adding the `--report` flag, which makes it possible to keep a record of an analysis or to attach one to written work. The flag was used on a Caesar encryption and the generated file was then listed and opened to confirm its contents.

```
python main.py caesar-encrypt --text "ATTACK AT DAWN" --shift 3 --report
ls -l reports
cat reports/analysis_<timestamp>.txt
```

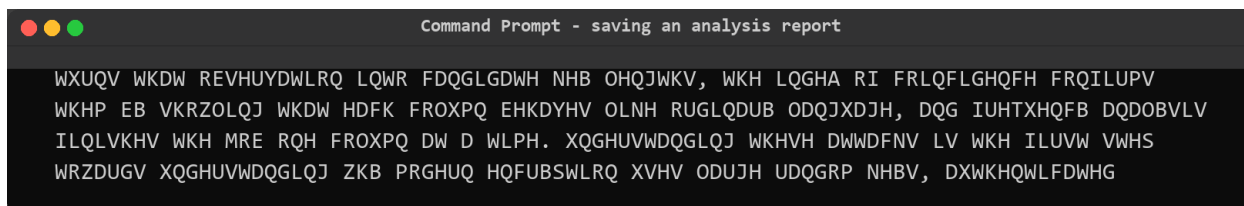


```
Command Prompt - saving an analysis report

$ python main.py crack-caesar --input sample_data/caesar_sample.txt --report
CAESAR CRACKING
-----
Ciphertext length      : 1287
Candidates evaluated   : 26
Candidates shown       : 5
Ciphertext: KHOOR ZRUOG

FUBSWRJUDSKB LV WKH SUDFWLFH DQG VWXGB RI WHFKQLTXHV IRU VHFXUH FRPPXQLFDWLRQ LQ WKH
SUHVVHQFH RI DGYHUVLHV. FODVVLFO FLSKHUV VXFK DV WKH FDHVDU FLSKHU DQG WKH YLJHQHUH
FLSKHU DUH QRW VHFUXH, EXW WKHB UHPDLQ HAFHOHQW WHDFKLQJ WRROV EHFDXVH HYHUB VWHS RI
WKHLU FUBSWDQDOBVLV FDQ EH SHUIRUPHG EB KDQG. D FDHVDU FLSKHU VKLIWV HDFK OHWWHU EB D
ILAHG DPRXQW, VR WKHUH DUH RQOB WZHQWB VLA SRVVLEOH NHBV DQG DQB DWWDFNVH FDQ WHVW WKHP
DOO. D YLJHQHUH FLSKHU XVHV D UHSHDWLQJ NHBZRUG WR DSSOB D GLIIHUVHQW VKLIW WR HYHUB
OHWWHU, ZKLFK KLGHV WKH VLPSON OHWWHU IUHTXHQFLHV WKDW PDNH WKH FDHVDU FLSKHU VR HDVB
WR EUHDN. QHYHUWKHOHVV, WKH UHSHDWLQJ NHBZRUG LQWURGXFHV LWV RZQ ZHDNQHVV: ZKHQYHU WKH
NHB UHDOHQW ZLWK WKH SODLQHWAW WKH VDPH FLSKHUHWAW SDWWHUQ DSSHUV DJDLQ, DQG WKH
GLVWDQFH EHWZHHQ WKRVH UHSHWLWLRLQV LV D PXOWLSOH RI WKH NHB OHQJWK. NDVLVNL HADPLQDWLRQ
```

Figure 46: Saving a report. After the encryption output the tool confirms that the report was written to `reports/analysis_<timestamp>.txt`, and the directory listing shows the newly created file with its timestamped name.



```
Command Prompt - saving an analysis report

WXUQV WKDW REVHUYDWLRQ LQWR FDQGLGDWH NHB OHQJWKV, WKH LQGHQ RI FRLQLGHQFH FRQILUPV
WKHP EB VKRZOLQJ WKDW HDFK FROXPQ EHKDYH OLNH RUGLQDUB ODQJXDJH, DQG IUHTXHQFB DQDOBVLV
ILQLVKHV WKH MRE RQH FROXPQ DW D WLPH. XQGHUVWDQGLQJ WKHVH DWWDFNV LV WKH ILUVW VWHS
WRZDUGV XQGHUVWDQGLQJ ZKB PRGHUQ HQFUBSWLRQ XVHV ODUJH UDQGRP NHBV, DXWKHQWLFDWHG
```

Figure 47: Saving a report (continued).

```

Text file - generated analysis report

$ cat $(ls -t reports/*.txt | head -1) # newest generated report
=====
CRYPTOGRAPHIC CIPHER ANALYSIS REPORT
=====

Date: 2026-09-21 12:08:53
Cipher: Caesar Cipher
Operation: Automatic cracking

Input statistics:
-----
Ciphertext length      : 1287
Alphabetic characters : 1071

Input:

```

Figure 48: The generated report file. It carries a header with the date, cipher and operation, an input statistics block, the input and output text, the educational notes and the security assessment of the cipher – a self-contained record that can be submitted as evidence of the work.

```

Text file - generated analysis report

-----
KHOOR ZRUOG

FUBSWRJUDSKB LV WKH SUDFWLFH DQG VWXGB RI WHFKQLTXHV IRU VHFXUH FRPPXQLFDWLRQ LQ WKH
SUHVHQFH RI DGYHUVLHV. FODVLFDO FLSKHUV VXFK DV WKH FDHVDU FLSKHU DQG WKH YLJHQHUH
FLSKHU DUH QRW VHFXUH, EXW WKHB UHPDLQ HAFHOHQW WHDFKLQJ WRROV EHFDXVH HYHUB VWHS RI
WKHLU FUBSWDQDOBVLV FDQ EH SHUIRUPHG EB KDQG. D FDHVDU FLSKHU VKLIW HDFK OHWWHU EB D
ILAHG DPRXQW, VR WKHUH DUH RQOB WZHQW VLA SRVLEOH NHBV DQG DQB DWWDFNHU FDQ WHVW WKHP
DOO. D YLJHQHUH FLSKHU XVHV D UHSHDWLQJ NHBZRUG WR DSSOB D GLIIHQW VKLIW WR HYHUB
OHWWHU, ZKLFK KLGHV WKH VLPSON OHWWHU IUHTXHQFLHV WKDW PDNH WKH FDHVDU FLSKHU VR HDVB
WR EUHDN. QHYHUWKHOVV, WKH UHSHDWLQJ NHBZRUG LQWURGXFHV LWV RZQ ZHDNQHV: ZKHQHYHU WKH
NHB UHDOHQW ZLWK WKH SODLQWHAU WKH VDPH FLSKHUWHAU SDWWHUQ DSSHUV DJDLQ, DQG WKH
GLVWDQFH EHWZHHQ WKRVH UHSHDWLQJ LV D PXOWLSOH RI WKH NHB OHQJWK. NDVLNL HADPLQDWLRQ
WXUQV WKDW REVHUYDWLRQ LQWR FDQGLGDWH NHB OHQJWKV, WKH LQGHU RI FRLQFLGHQFH FRQILUPV
WKHP EB VKRZOLQJ WKDW HDFK FROXPQ EHKDYHV OLNH RUGLQDUB ODQJXDJH, DQG IUHTXHQFB DQDOBVLV
ILQLVKHV WKH MRE RQH FROXPQ DW D WLP. XQGHUVWDQGLQJ WKHVH DWDFNV LV WKH ILUVW VWHS
WRZDUGV XQGHUVWDQGLQJ ZKB PRGHUQ HQFUBSWLRQ XVHV ODUJH UDQGRP NHBV, DXWKHQWLFDWHG
FLSKHUWHAU DQG FDUHIXOB UHYLHZHG DOJRULWKPV LQVWHDG RI VKDUHG VHFUHV.

```

Figure 49: The generated report file (continued).

```

Text file - generated analysis report

Candidate solutions (estimated):
-----
1. shift 3  score 70.52  HELLO WORLD

CRYPTOGRAPHY IS THE PRACTICE AND STUDY OF TECHNIQUES FOR SECURE COMMUNICATION IN THE

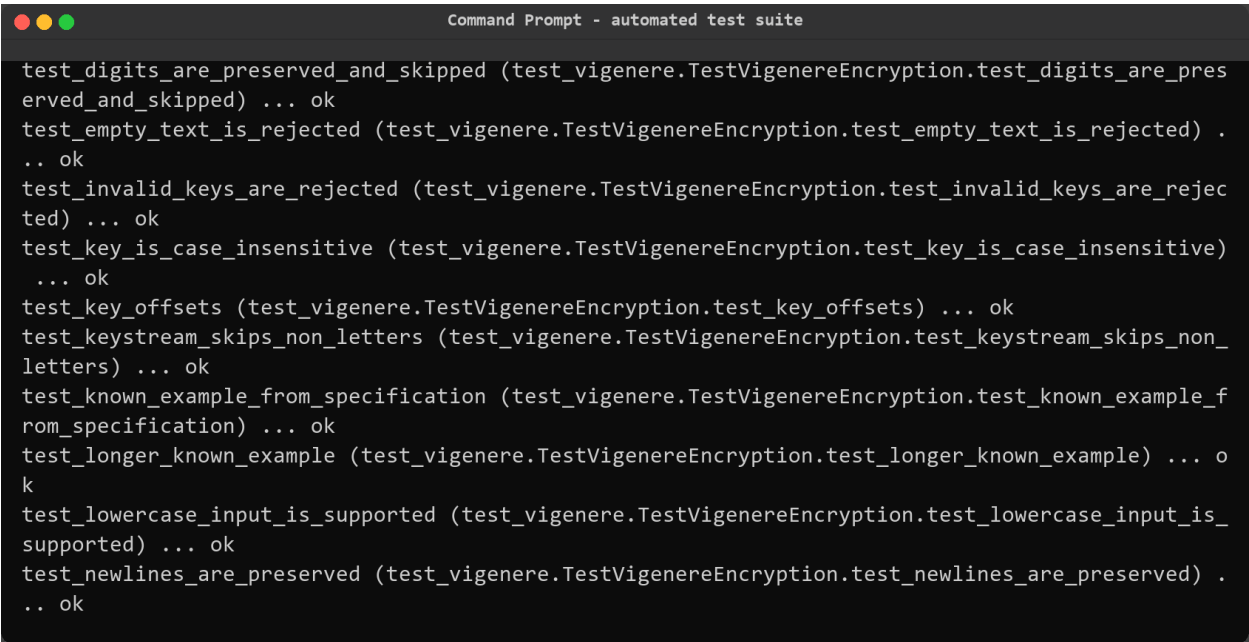
```

Figure 50: The generated report file (continued).

Step 14 – Running the automated test suite

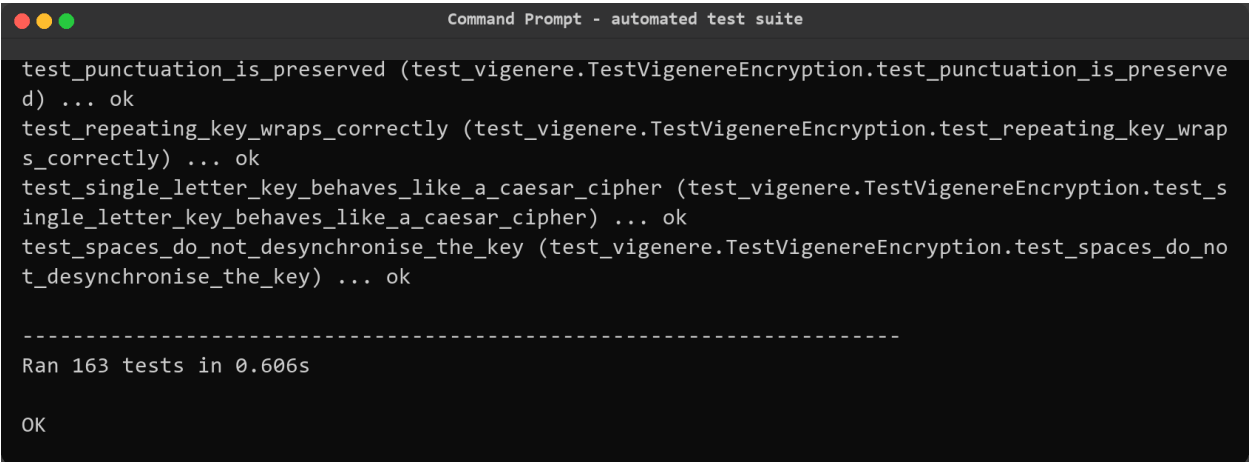
The last step was to confirm that none of the demonstrations above depended on manual testing alone. The project ships with 163 unit and integration tests covering the cipher implementations (including the known textbook examples, punctuation, digits and case handling), the cryptanalysis algorithms, the validators, the file operations and the command line interface itself. The suite was run in verbose mode so that individual test names as well as the final summary are visible.

```
python -m unittest discover -s tests -v
```



```
test_digits_are_preserved_and_skipped (test_vigenere.TestVigenereEncryption.test_digits_are_preserved_and_skipped) ... ok
test_empty_text_is_rejected (test_vigenere.TestVigenereEncryption.test_empty_text_is_rejected) .. ok
test_invalid_keys_are_rejected (test_vigenere.TestVigenereEncryption.test_invalid_keys_are_rejected) ... ok
test_key_is_case_insensitive (test_vigenere.TestVigenereEncryption.test_key_is_case_insensitive) ... ok
test_key_offsets (test_vigenere.TestVigenereEncryption.test_key_offsets) ... ok
test_keystream_skips_non_letters (test_vigenere.TestVigenereEncryption.test_keystream_skips_non_letters) ... ok
test_known_example_from_specification (test_vigenere.TestVigenereEncryption.test_known_example_from_specification) ... ok
test_longer_known_example (test_vigenere.TestVigenereEncryption.test_longer_known_example) ... ok
test_lowercase_input_is_supported (test_vigenere.TestVigenereEncryption.test_lowercase_input_is_supported) ... ok
test_newlines_are_preserved (test_vigenere.TestVigenereEncryption.test_newlines_are_preserved) .. ok
```

Figure 51: End of the automated test run. The last tests exercise the Vigenere implementation – key offsets, keystream behaviour, repeating keys and the known example – and the summary reports that all 163 tests ran and returned OK.



```
test_punctuation_is_preserved (test_vigenere.TestVigenereEncryption.test_punctuation_is_preserved) ... ok
test_repeating_key_wraps_correctly (test_vigenere.TestVigenereEncryption.test_repeating_key_wraps_correctly) ... ok
test_single_letter_key_behaves_like_a_caesar_cipher (test_vigenere.TestVigenereEncryption.test_single_letter_key_behaves_like_a_caesar_cipher) ... ok
test_spaces_do_not_desynchronise_the_key (test_vigenere.TestVigenereEncryption.test_spaces_do_not_desynchronise_the_key) ... ok

-----
Ran 163 tests in 0.606s

OK
```

Figure 52: End of the automated test run (continued).

Reproducing this section: run the commands inside `crypto_cipher_analyzer` with Python 3.10 or later. All twenty-four screenshots were produced from these exact commands, and the captured console output is stored as plain text under `report_assets/captures` so that every figure can be checked against it.

SECTION 5 – RESULTS & OBSERVATIONS

5. Results and Observations

The tool and the sample data produced a complete end-to-end demonstration: both ciphers encrypt and decrypt correctly, the Caesar cipher was broken automatically, the Vigenere sample was fingerprinted, sized and cracked back to its keyword, and file based round trips were lossless. The findings below summarise what was observed, how each was measured and how much it matters for the security of the two algorithms.

5.1 Summary of Findings

#	Finding / Observation	Details	Severity / Impact
1	The Caesar cipher is broken instantly	All 26 shifts were scored with chi-square; shift 3 ranked first with 70.52 against 22.94 for the runner-up and produced readable English. No plaintext hint was needed.	Critical
2	A monoalphabetic cipher preserves the letter distribution	In the Caesar sample H occurred 15.31% of the time against 6.09% in English while E fell to 1.12%, and the Index of Coincidence was 0.06987 against 0.0667 for English prose.	High
3	The Vigenere cipher flattens the distribution	The overall Index of Coincidence of the Vigenere sample was 0.04351, close to the 0.0385 expected from random text, so single-letter frequency analysis finds nothing.	High
4	Repeated sequences leak the key length	50 repeated fragments were found; OWCSI and WCSID each occurred three times at distances of 500 and 395, both divisible by 5, so key length 5 ranked first.	High
5	The keyword was recovered automatically	Column-by-column analysis suggested L, E, M, O and N, and the assembled key LEMON ranked first with a fitness score of 70.15, decrypting the file into plain English.	Critical
6	Statistical evidence must be verified, not trusted	Key length 2 collected almost as many Kasiski hits (160) as length 5 (164) but gave a column IC of only 0.04341 and meaningless output, so the combination of evidence and reading the result is what settles the answer.	Medium
7	File operations are lossless and non-destructive	1287 characters were read and 1287 written in both directions, the original sample files were left untouched and an existing output file was never replaced without explicit confirmation.	Informational

5.2 Analysis

The results show a clear progression from trivial to subtle, and each stage depends on the one before it. The Caesar cipher fails on its first weakness, the size of the key: with twenty-six possible keys an exhaustive search costs nothing, and the chi-square score merely automates the human step of deciding which of the twenty-six candidates looks like English. The frequency analysis explains why

that works. Because the shift maps every letter of the plaintext to exactly one letter of the ciphertext, the shape of the distribution is preserved and simply slides along the alphabet; the Index of Coincidence stays close to the English value and confirms that a single alphabet was used, which is the fingerprint of a monoalphabetic cipher.

The Vigenere cipher defeats that specific attack but not the underlying idea. Its overall Index of Coincidence of 0.04351 is barely above the value for random text, so counting letters in the whole ciphertext reveals nothing. What it cannot hide is repetition. Whenever the keyword realigns with the plaintext, the same plaintext fragment is encrypted with the same part of the key, so exact repeats appear at distances that are multiples of the key length. Both observations in the sample – OWCSI and WCSID repeating 500 and 395 letters apart, and both distances divisible by 5 – pointed straight at a key length of five. The tool did not act on that alone: it also measured the average Index of Coincidence of the columns for every candidate length, because when the guessed length is right each column contains one consistent Caesar shift and therefore looks like ordinary language again. Key length 5 scored 0.07079 on that measure while the equally well supported length 2 scored only 0.04341, and combining both kinds of evidence is what pushed 5 to the top.

Once the length was fixed at five, the cipher collapsed into five independent Caesar ciphers and the rest followed mechanically. Each column was scored against English letter frequencies, the best three letters were suggested for each position and complete keys were assembled from those suggestions. L, E, M, O and N emerged with scores between 50 and 58 against alternatives in the low twenties, and the full key LEMON scored 70.15 and decrypted the whole file into readable prose. Two lessons from the results are worth stating explicitly. First, a large keyspace does not make a cipher strong: 26^5 is more than eleven million keys, which is far beyond brute force by hand, yet the key fell to statistics in a fraction of a second because the cipher leaks structure rather than because the key was guessed. Second, statistical output must be read critically. Key length 2 had almost as much Kasiski support as key length 5, and only the column Index of Coincidence and the readability of the decrypted text separated them, which is why the tool labels every estimate as an estimate and always prints the decrypted preview beside the candidate key.

The security assessments close the loop with the theory. The report for the Vigenere cipher says directly that its keyspace is not the weakness, and the experiments above are the evidence for that statement. Both ciphers share weaknesses that no amount of care can fix: they are deterministic, so identical plaintext always produces identical ciphertext; they provide no integrity protection, so ciphertext can be altered without detection; and they share a single secret key, so the problem of delivering the key is never solved. Those limitations are exactly what modern cryptography replaced with large random keys, authenticated encryption and reviewed algorithms, which is why the tool's verdict for both ciphers is educational use only.

SECTION 6 – CHALLENGES FACED

6. Challenges Faced

Several problems came up during the build and during the demonstrations. They are recorded here together with the solution that was adopted, because most of them changed the design of the tool.

Challenge Faced	How I Solved It
The cryptanalysis output is statistical, and it is easy to present a guess as a confirmed result.	Every estimated value is labelled in the output itself – 'Candidate key lengths (estimated)', 'BEST GUESS (unconfirmed)' – the decrypted preview is always printed next to a candidate key so that it can be judged by reading it, and the notes state plainly that estimates must be verified.
The interactive menu blocked the automated tests, because <code>input()</code> waits forever when the tool is fed from a pipe or a file.	The prompt helper catches <code>EOFError</code> , sets an end-of-input flag and returns the default, and the menu checks that flag after every operation, so a piped session exits cleanly instead of looping.
The accented letter in 'Vigenere' crashed the tool on a console using a legacy Windows code page.	Output streams are reconfigured with <code>errors='replace'</code> at start-up, so an unencodable character degrades gracefully instead of raising <code>UnicodeEncodeError</code> , and colours are disabled automatically when the output is not an interactive terminal.
Kasiski examination produced false positives: a short repeated fragment can appear by chance and its distance may be divisible by an innocent number.	Repeated sequences are reported only from a minimum length of three characters, the factors are ranked by the amount of evidence rather than by a single match, and the result is combined with the column Index of Coincidence before any key length is used.
Some commands produce more output than fits on one page, which made the first screenshots unreadable in the report.	Long outputs were split into consecutive figures at natural boundaries (header, evidence tables, recovery, best guess) and long lines are soft-wrapped at the width of the window shown, so the text in every figure stays legible.
A file operation could overwrite the very file it was reading from.	The file handler refuses to write unless the destination is different or <code>--overwrite</code> is given, the default output name adds the suffix <code>_out</code> , and the interactive mode asks for confirmation before replacing anything.

SECTION 7 – LEARNING OUTCOMES

7. Learning Outcomes

Working through this project produced the following concrete skills and insights:

- I can explain why the Caesar cipher collapses under an exhaustive search and implement the search myself, scoring candidates with the chi-square statistic instead of relying on a single frequency comparison.
- I understand the difference between a monoalphabetic and a polyalphabetic cipher in measurable terms: the Caesar sample held an Index of Coincidence of 0.06987 close to English, while the Vigenere sample fell to 0.04351 close to random text.
- I can carry out a Kasiski examination by hand on a short ciphertext – finding repeated fragments, measuring the distances, factorising them and reading the shared factors as candidate key lengths – and I know that the result is a hypothesis rather than a fact.
- I understand why guessing the key length correctly makes the columns look like ordinary language again, which is exactly what the average Index of Coincidence per key length measures, and how that test separated key length 5 (0.07079) from key length 2 (0.04341).
- I can recover a Vigenere keyword column by column by treating every column as its own Caesar cipher, and I verified the recovered key by reading the decrypted text rather than by trusting the score.
- I learned why a large keyspace is not the same thing as security: 26^5 is over eleven million keys, yet the Vigenere sample was cracked from statistics alone, which is the lesson that carries over to modern cryptography.
- On the engineering side I practised structuring a program into focused packages, writing defensive input validation, keeping interactive, command line and test driven use of the same algorithms in sync, and backing the whole tool with 163 automated tests.

SECTION 8 – CONCLUSION & FUTURE SCOPE

8. Conclusion

The project met its objectives. A single self-contained Python tool now implements the Caesar and Vigenere ciphers, breaks the first by exhaustive search with chi-square scoring, and breaks the second by combining frequency analysis, the Index of Coincidence and Kasiski examination. Run against the supplied sample data it recovered the Caesar shift of 3 automatically and, with no knowledge of the plaintext, reduced the Vigenere sample to an estimated key length of 5 and the exact keyword LEMON as the highest ranked candidate. Both ciphers were also assessed for strength and reported as unsuitable for protecting real information.

The most valuable result is not the recovered key itself but the demonstration of how security is lost. The Caesar cipher failed because its keyspace is tiny; the Vigenere cipher had a keyspace of more than eleven million keys for a five-letter keyword and failed anyway, because a repeating key leaves statistical traces – repeated sequences at regular distances and columns that behave like plain language once the length is known. The project also showed how easy it is to mistake a statistical estimate for a proof: key length 2 had almost the same Kasiski evidence as the correct length 5 and was rejected only after its column Index of Coincidence and its decrypted output were examined. That habit of verifying evidence, which the tool enforces by labelling every guess as a guess and printing the decrypted text beside the candidate key, is the most transferable outcome of the exercise.

If the project were repeated, the cryptanalysis engine would be separated cleanly from the presentation layer from the start, and a non-interactive batch mode would be added early so that a whole folder of sample files could be analysed and compared in one run instead of one command at a time.

8.1 Future Scope

The following extensions would build naturally on what already exists:

- Add further classical ciphers (affine, Playfair, columnar transposition, one-time pad) and reuse the existing frequency and Index of Coincidence modules to attack them.
- Implement the Friedman test and mutual Index of Coincidence so that key length estimation no longer depends on repeated sequences alone.
- Add a graphical interface (Tkinter or a small web front end) that displays the letter frequency chart and the candidate ranking side by side for teaching purposes.
- Add multi-file batch analysis with a summary table so that several ciphertexts can be compared in one run.
- Compare the classical ciphers against a modern reference implementation of AES to show the transition from key secrecy to algorithm transparency.
- Support non-English alphabets and language profiles, so that the frequency and fitness scoring can be applied to text in other languages.

SECTION 9 – REFERENCES

9. References

- [1] Caesar cipher – Wikipedia. https://en.wikipedia.org/wiki/Caesar_cipher
- [2] Vigenere cipher – Wikipedia. https://en.wikipedia.org/wiki/Vigen%C3%A8re_cipher
- [3] Kasiski examination – Wikipedia. https://en.wikipedia.org/wiki/Kasiski_examination
- [4] Index of coincidence – Wikipedia. https://en.wikipedia.org/wiki/Index_of_coincidence
- [5] Chi-squared test – Wikipedia. https://en.wikipedia.org/wiki/Chi-squared_test
- [6] Python 3.13 documentation – Standard library reference (argparse, dataclasses, pathlib, statistics, unittest). <https://docs.python.org/3/library/>
- [7] python-docx documentation – reading and writing Microsoft Word files. <https://python-docx.readthedocs.io/>
- [8] Labmentix Cybersecurity Training Program – course notes and 15-day project brief (internal training material).

DECLARATION

Student Declaration

I hereby declare that this project report is my own original work, completed as part of the Labmentix Cybersecurity Training Program. The work presented in this report is genuine, and all external references have been duly acknowledged.

All commands, results and screenshots in Section 4 were produced by running the tool that accompanies this report on the sample data supplied with the project; the console output behind every figure is retained in the project folder for verification.

 Student Signature	Date of Submission
--	--------------------